

Structural Estimation in Corporate Finance: Project Plan

Zhaoxuan Wang

Version: June 25, 2026

Contents

1	Introduction	3
1.1	Overview	3
1.2	What business problems we solve	3
1.3	What is the core engine of the product	3
1.4	How to use this document	3
2	Corporate Finance Model	4
2.1	Risky debt model	4
3	Model Solvers	9
3.1	Howard's policy iteration	10
3.2	Neural policy iteration	17
3.3	Third solver (to be specified)	28
3.4	Network refinement on discrete grids	29
4	Model Estimation	32
4.1	Simulated Method of Moments (SMM)	32
4.2	Bayesian estimation	33
5	Validation and Diagnostics	34
5.1	Benchmark solver	34
5.2	Oracle test 1: Monte Carlo recovery of parameters and moments	35
5.3	Oracle test 2: policy network against benchmark	36
5.4	Economic property tests	37
5.5	Regression tests	38
5.6	Value and policy accuracy against VFI	39
5.7	Monte Carlo recovery results	39
6	Commercial Product, Governance, and Practical Considerations	44
6.1	Product 1: Corporate Finance Advisory (CFA)	44
6.2	Product 2: Debt Capital Markets (DCM)	46
6.3	Product 3: Leveraged Finance and Private Credit	47
6.4	Model risk and deployment issues	48
A	Regulatory Compliance Map	51
A.1	The shared compliance baseline and key takeaways	51
A.2	Consolidated list	52

A.3 Watch list	53
References	54
Appendix: Interview QA	55

Chapter 1

Introduction

1.1 Overview

1.2 What business problems we solve

1.3 What is the core engine of the product

1.4 How to use this document

Chapter 2

Corporate Finance Model

2.1 Risky debt model

The baseline model is a dynamic corporate finance model with leverage, investment, cash, endogenous default, costly equity issuance, and non-convex adjustment costs. A firm observe states (capital, debt, productivity) today, choose actions (investment, leverage, saving) to maximize firm's present value, under rational expectation of future uncertainty (stochastic productivity shocks). The model has a partial equilibrium structure: the bond price is set by the breakeven condition of a rational lender who takes into account the firm's default probability over next-period uncertainty.

The model representation below is adopted from [Duarte and Fonseca \(2026\)](#). This is the same risky debt model class of [Strebulaev and Whited \(2012, Sec. 3.6\)](#), and can be extended to a general equilibrium version of [Gao et al. \(2021, Sec. 3.2\)](#) where the risk-free interest rate clears household savings in the economy.

2.1.1 Environment and variables

Time is discrete. An infinitely-lived firm combines capital and labor to produce output using a stochastic technology subject to productivity shocks. Before production, it pays fixed operating costs and finances with current profit flows, risky debt, external equity, and internal cash. There are two frictions associated with raising external financing: defaulting on debt incurs deadweight costs, and equity issuance incurs fees.

States vector: $(z, k, b - c)$

- z : productivity
- k : capital stock
- $b - c$: debt net of cash, where both gross debt b and cash c were chosen in previous period.

Action vector: (i, b', c')

- i : investment rate, defined so that $k' = (1 + i - \delta)k$. Investment level is $I = i \cdot k$
- b' : next-period gross debt, per unit of next-period capital k'
- c' : next-period cash, per unit of next-period capital k'

The next-period net-debt state is therefore $b' - c'$. A net cash position is when $b' - c' < 0$. Continuation value in Bellman equation is evaluated at $(z', k', b' - c')$.

2.1.2 Technology and production

Output is produced with capital k and labor l :

$$y = z(k^\alpha l^{1-\alpha})^\theta \quad (2.1)$$

where $\alpha \in (0, 1)$ is capital share, $\theta \in (0, 1)$ governs returns to scale. Stochastic productivity $\log z$ follows an AR(1) process:

$$\log z' = \rho \log z + \sigma \varepsilon', \quad \varepsilon' \sim \mathcal{N}(0, 1). \quad (2.2)$$

Labor is flexible and hired at competitive wage w . Substituting its closed-form optimum gives the reduced-form operating surplus:

$$\pi(z, k) = z A_\pi k^\xi - c_f, \quad \xi \equiv \frac{\alpha\theta}{\nu}, \quad \nu \equiv 1 - (1 - \alpha)\theta. \quad (2.3)$$

In partial equilibrium the wage is normalized to $w = 1$. Set marginal product of labor $w = \partial y / \partial l$ and A_π is derived as a constant in (α, θ) :

$$A_\pi = \nu \left((1 - \alpha)\theta \right)^{\frac{(1-\alpha)\theta}{\nu}}.$$

Hence, labor and wage do not enter the firm's dynamic problem.

2.1.3 Investment and adjustment costs

Capital accumulates as

$$k' = (1 - \delta)k + I.$$

Investment incurs a convex cost $\frac{1}{2}\gamma_1 I^2/k$ and a fixed cost $\gamma_0 k \mathbf{1}\{I > 0\}$ that is proportional to capital and is paid whenever investment is positive:

$$\psi(I, k) = \frac{1}{2}\gamma_1 \frac{I^2}{k} + \gamma_0 k \mathbf{1}\{I > 0\}.$$

2.1.4 Debt, default, recovery, and bond pricing

The firm issues one-period debt at an endogenous price q that reflects default risk. Let uppercase letter denote levels of cash and debt, and lowercase letters are the same quantity in per unit capital:

$$C = c \cdot k, \quad B = b \cdot k, \quad C' = c' \cdot k', \quad B' = b' \cdot k'$$

where c is cash saving ratio and b is leverage ratio, both normalized by capital stock.

Liquidation recovery. A defaulting firm is liquidated, and lenders recover

$$R(k', c') = c'k' + \chi(1 - \delta)k',$$

its cash plus the salvage value of depreciated capital, where χ is the recovery rate.

This recovery formula is different from that in [Strebulaev and Whited \(2012, Sec. 3.6\)](#) where the defaulting firm is liquidated after one more production, and then lender recovers a fraction of the output $\pi(k', z')$. In current formula, the liquidation happens when z' is realized and firm choose to default if equity value goes negative, without any further production.

Default rule. The firm defaults when (i) its continuation equity value is negative and (ii) its liquidation proceeds do not cover its outstanding debt. With continuation equity value $V(z', k', b' - c')$, recovery R , and outstanding gross debt $b'k'$, the default indicator is

$$\mathbf{1}_{\text{def}} = \mathbf{1}\{V(z', k', b' - c') < 0\} \cdot \mathbf{1}\{R(k', c') < b'k'\}. \quad (2.4)$$

Condition (i) is the single default indicator used in [Strebulaev and Whited \(2012, Sec. 3.6\)](#). The default boundary as a function of z' tomorrow, when $(k', b' - c')$ were already chosen today.

Condition (ii) add one more realistic constraint, $R < b'k'$ (equivalently $c' + \chi(1 - \delta) < b'$), which depends only on the actions and parameters, not on z' . It is a *deterministic* gate: it does not constrain i , b' , or c' , and it keeps the recovery rate below one whenever default occurs.

Bond price. $q(\cdot)$ is endogenous and satisfies the break-even condition:

$$b' \cdot q(z, k', c', b') = \frac{1}{1 + r_f(1 - \tau)} \mathbb{E}_{z'|z} \left[(1 - \mathbf{1}_{\text{def}})b' + \mathbf{1}_{\text{def}} \frac{R(k', c')}{k'} \right]. \quad (2.5)$$

Define the default probability as:

$$P_{\text{def}} = \Pr(V(z', k', b' - c') < 0) = \mathbb{E}_{z'|z} [\mathbf{1}\{V(z', k', b' - c') < 0\}].$$

Since the recovery ratio $R(k', c')/(b'k')$ are constant across z' , we can rewrite the bond price as

$$q = \frac{1}{1 + r_f(1 - \tau)} \left[(1 - \mathbf{1}\{R < b'k'\}) \cdot P_{\text{def}} + \mathbf{1}\{R < b'k'\} \cdot P_{\text{def}} \frac{R(k', c')}{b'k'} \right]. \quad (2.6)$$

This bond pricing equation 2.6 is useful for implementation because $\mathbf{1}\{R < b'k'\}$ is a deterministic gate given (k', b', c') . The only remaining object is default probability P_{def} , which requires an approximation of V and an unbiased estimator for $\mathbb{E}_{z'|z} [\mathbf{1}\{V(z', k', b' - c') < 0\}]$ across z' .

When $R \geq b'k'$, the liquidation recovery can cover debt repayment, so bond is risk-free $q = \frac{1}{1 + r_f(1 - \tau)}$. When $R < b'k'$, the recovery rate is below one and the bond prices in the default risks.

Tax benefit of debt. The tax benefit of debt is capitalized into the bond price, where the discount has $(1 - \tau)$. The firm's promised repayment is discounted at the after-tax rate $r_f(1 - \tau) < r_f$.

2.1.5 Dividends and equity issuance

Cash flow is split into a preproduction and a postproduction stage within each period. This makes the model timing explicit and avoids confusion.

Pre-production net cash flow after firm choose (i, b') :

$$d_1 = q \cdot b' \cdot k' - b \cdot k - c_f \quad (2.7)$$

where the firm issues new debt $b' \cdot k'$ at price q , repays old debt $b \cdot k$, and pays the fixed cost c_f .

If $d_1 > 0$, the firm rolls over its debt with positive cash surplus. This excess cash carries forward to the next stage. If $d_1 < 0$, the firm faces a pre-production deficit. Because it has no cash savings or revenues yet, it is forced to issue equity with cost:

$$\eta(d_1) = \lambda_0 k + \lambda_1 |d_1|$$

Post-production net cash flow after firm choose saving c' :

$$d_2 = z A_\pi k^\xi + \max\{d_1, 0\} - i \cdot k - c' k' \iota_c - \psi(I, k) \quad (2.8)$$

where $\psi(I, k)$ is capital adjustment cost, $I = i \cdot k$ is investment chosen by firm. The opportunity cost of carrying cash is $\iota_c = 1/(1 + r_c r_f (1 - \tau))$. Current model assumes that cash earns no interest or tax benefit, so $r_c = 0$ and $\iota_c = 1$.

If $d_2 > 0$ the firm distributes this to shareholders (dividend payout). If $d_2 < 0$ it issues equity with the same cost function $\eta(d_2)$.

Net payout to shareholders combines two cash flows:

$$D = d_2 + d_1 \cdot \mathbf{1}\{d_1 < 0\} - (\lambda_0 k + \lambda_1 |d_1|) \cdot \mathbf{1}\{d_1 < 0\} - (\lambda_0 k + \lambda_1 |d_2|) \cdot \mathbf{1}\{d_2 < 0\}. \quad (2.9)$$

In this equation, d_2 already includes any positive surplus from debt rollover before production, as well as the net revenue from production after investment and saving choices. The negative term $d_1 \cdot \mathbf{1}\{d_1 < 0\}$ reflects that shareholders had to inject money to cover the pre-production deficit. The last two terms reflect deadweight losses associated with equity issuance, which directly reduce the net payout to shareholders.

If $D > 0$, the firm pays a positive net dividend to shareholders. If $D < 0$, the firm is in a net equity issuance position, meaning shareholders are net contributors of capital to the firm during this period.

2.1.6 Policy, value, and Bellman equation

A deterministic policy maps the state to the three controls:

$$h(z, k, b) = (h^i(z, k, b), \quad h^b(z, k, b), \quad h^c(z, k, b)). \quad (2.10)$$

On-policy value function. $V^h(z, k, b)$ is the firm's equity value of following the policy h forever. It is the unique fixed point of

$$V^h(z, k, b) = D(z, k, b, i, b', c') + \frac{1}{1+r_f} \mathbb{E}_{z'|z} [\max\{V^h(z', k', b' - c'), 0\}],$$

where the controls are set by the policy, $i = h^i(z, k, b)$, $b' = h^b(z, k, b)$, $c' = h^c(z, k, b)$, with $k' = (1 + i - \delta)k$ and next-period net debt $b' - c'$. The map is a contraction, so V^h exists and is unique.

Bellman equation. The optimal value V^* satisfies

$$V^*(z, k, b) = \max_{i, b', c'} \left\{ D(z, k, b, i, b', c') + \frac{1}{1+r_f} \mathbb{E}_{z'|z} [\max\{V^*(z', k', b' - c'), 0\}] \right\} \quad (2.11)$$

over feasible (i, b', c') , with $k' = (1 + i - \delta)k$. The inner $\max\{V', 0\}$ is limited liability: after z' is realized, shareholders walk away when equity value is negative.

Optimal policy. The optimal policy is the maximizer of the Bellman right-hand side, evaluated at the optimal continuation value V :

$$h^*(z, k, b) = \arg \max_{i, b', c'} \left\{ D(z, k, b, i, b', c') + \frac{1}{1+r_f} \mathbb{E}_{z'|z} [\max\{V^*(z', (1+i-\delta)k, b'-c'), 0\}] \right\}. \quad (2.12)$$

By construction $V^* = V^{h^*}$ and $V^* \geq V^h$ for every policy h .

Proposition 2.1: Model Solution

The solution of the risky debt model is the triple, a value function $V^*(z, k, b)$, a policy function $h^*(z, k, b)$, and a bond price $q^*(z, k', b', c')$, that jointly satisfy the endogenous default rule (2.4), the bond pricing equation (2.6), and the Bellman equation (2.11), at every state (z, k, b) and every feasible action (i, b', c') .

Note that the model solution (V^*, h^*, q^*) in Proposition 2.1 is *not* necessarily unique, because V and q is coupled fixed point, and thus the Bellman map $V = T(V, q(V))$ is not obviously a contraction. The remainder of the paper omits the star and writes V and h for the optimal V^* and h^* .

Chapter 3

Model Solvers

A model solver is an algorithm that inputs model parameter values, state-action space bounds, and parameter bounds; apply the algorithm; and output the triplet of optimal value function, policy function, and bond price function defined in Proposition 2.1:

This chapter describes the algorithms and implementation details of three model solvers:

- Howard’s policy iteration on discrete grids ([Howard, 1960](#))
- Neural policy iteration from [Duarte and Fonseca \(2026\)](#) and [Duarte et al. \(2024\)](#).
- Euler residual minimization from [Wong \(2026\)](#), extended from [Maliar et al. \(2021\)](#)

There exists other viable model solvers not discussed here, so the list is not exhasutive.

The *Howard’s policy iteration* is primarily used as a trusted benchmark to validate the solution of other solvers on discrete grids. Its main advantage is the mathematical guarantee for a correct solution. The algorithm will converge to a particular coupled fixed point that jointly satisfies the risky debt model solution defined in Proposition 2.1. However, this algorithm is less useful for the full estimation pipeline because (i) the curse of dimensionality; (ii) grid approximation error for smooth objects; and (iii) each model solving is expensive and cannot be amortized.

The *neural policy iteration* is used as the default solver for the risky debt model in this project. The algorithm is adopted from “Block 1” of the full estimation pipeline in [Duarte and Fonseca \(2026, Section 3.1\)](#). The goal is to train separate neural networks to approximate the *universal* value and policy function over state $\times$ parameter space. The method breaks the circularity between bond price and value using a separate target value network and a numerical approximation of the default probability. The core algorithm alternatives between two steps until convergence:

- Policy evaluation: fix the policy and bond price, train the value network to minimize the Bellman equation residual.
- Policy improvement: fix the value, train the policy network to maximize the right-hand-side of the Bellman equation; update bond price.

There is also a continuous-time version of this algorithm, see [Duarte et al. \(2024, Section 2\)](#).

The *Euler residual minimization* method is the third class of model solver. The idea is to explicitly write down the residuals measuring the violations to the first-order neccesary condition (Euler

equation), bond-pricing equation, equality and inequality constraints, and policy-maximization condition. The algorithm then jointly minimizes the residuals to learn neural network approximation of the optimal policy and value functions. This chapter describes the implementation from [Wong \(2026\)](#) that is specific to the risky debt model. For a general framework of this method, see [Maliar et al. \(2021, Section 3\)](#)’s Euler residual minimization method, and [Fernández-Villaverde \(2026, Section 8\)](#)’s Bellman-and-policy residual minimization method.

3.1 Howard’s policy iteration

The Howard’s *policy function iteration* solver computes the model solution (V, h, q) of Proposition 2.1 by discretizing the state and action spaces onto grids and iterating to the fixed point. It is the project’s *trusted benchmark*: an intentionally exact and slow “oracle” used to validate and refine the neural solutions, and is never used inside the estimation loop.

The algorithm alternates a greedy *policy improvement* step (search the whole action grid for the best action at each state) with an exact *policy evaluation* step (solve for the value of following that policy). The endogenous bond price is recomputed from the current value at the start of every round, so the value, policy, and bond price converge to a joint fixed point together.

3.1.1 Grids

Discretize the three state variables (z, k, b) and the three actions (i, b', c') . On the grid we enumerate the *next* capital k' directly instead of the investment rate i ; the two are equivalent through $k' = (1 + i - \delta)k$, so a chosen k' pins down $i = k'/k - (1 - \delta)$.

- **Productivity** $z \in \{z_1, \dots, z_{N_z}\}$: the N_z Tauchen nodes of the AR(1) for $\log z$, with transition matrix P whose entry $P_{jl} \equiv \Pr(z' = z_l \mid z = z_j)$ satisfies $\sum_l P_{jl} = 1$.
- **Capital state** $k \in \{k_1, \dots, k_{N_k}\}$: log-spaced over $[\underline{k}, \bar{k}]$, where $\underline{k} = k^*(z_{\min})$ and $\bar{k} = k^*(z_{\max})$ are the frictionless steady-state capitals at the lowest and highest productivity nodes.
- **Net-debt state** $b \in \{b_1, \dots, b_{N_b}\}$: uniform over $[\underline{b}, \bar{b}]$.
- **Next capital** $k' \in \{k'_1, \dots, k'_{N_{k'}}\}$: log-spaced over the same $[\underline{k}, \bar{k}]$.
- **Gross debt** $b' \in \{b'_1, \dots, b'_{N_{b'}}\}$: uniform over $[0, \bar{b}']$.
- **Cash** $c' \in \{c'_1, \dots, c'_{N_{c'}}\}$: uniform over $[0, \bar{c}']$.

Write a generic action as the triple (k', b', c') , ranging over the full product $\mathcal{A} = \{k'\} \times \{b'\} \times \{c'\}$ of size $N_{k'}N_{b'}N_{c'}$. The next-period net-debt state implied by an action is $b' - c'$, which is generally *off* the b grid.

Objects on the grid.

- $V(z_j, k_i, b_m)$: the firm’s equity value at each state. It is stored *unclamped*: a negative value marks a default state (the firm walks away), consistent with the limited-liability term $\max\{V, 0\}$ inside the Bellman expectation.
- $q(z_j; k', b', c')$: the endogenous bond price for current shock z_j and action (k', b', c') .
- $h(z_j, k_i, b_m) = (k', b', c')$: the policy, i.e. the chosen action at each state.

3.1.2 Building blocks evaluated on the grid

Fix the current value V . For a current shock z_j and an action (k', b', c') define the following objects, all of which feed the Bellman right-hand side.

Off-grid continuation value. The continuation equity value is evaluated at the next-period state $(\log z_l, \log k', b' - c')$. Since the action grids $(\log k', b' - c')$ are finer and need not lie on the state grids (k, b) , we evaluate the next-period equity value $\widehat{V}(\log z_l, \log k', b' - c')$ with bilinear interpolation of $\widehat{V}(z_l, \cdot, \cdot)$ in state grids $(\log k, b)$. Note that the capital grids are log-spaced so the actual interpolation use $\log k$. See section 3.1.5 for details.

Expected continuation. Using the Tauchen row P_j to take the expectation over next-period productivity, the expected continuation value is:

$$C(z_j; k', b', c') = \sum_{l=1}^{N_z} P_{jl} \max\{\widehat{V}(z_l, k', b' - c'), 0\}. \quad (3.1)$$

where C applies limited liability state-by-state.

Default probability. Holding the action (k', b', c') fixed, P_{def} integrates the default indicator over the next-period states $(z_l, k', b' - c')$ using the same Tauchen row P_j :

$$P_{\text{def}}(z_j; k', b', c') = \sum_{l=1}^{N_z} P_{jl} \mathbf{1}\{\widehat{V}(z_l, k', b' - c') < 0\}. \quad (3.2)$$

This is the expected share of defaulting next-period states. Under the Tauchen chain the conditional law of z' given z_j is exactly the row P_j , so the sum equals the model's conditional default probability $\mathbb{E}_{z'|z_j}[\mathbf{1}\{V(z', k', b' - c') < 0\}]$. Capital k' and net debt $b' - c'$ are fixed by the action, so the only thing averaged over is the productivity shock z' .

Bond price. With recovery $R(k', c') = c'k' + \chi(1 - \delta)k'$ and the deterministic default gate $g = \mathbf{1}\{R < b'k'\}$ (both from the default rule 2.4), the discretized bond price is the grid version of the bond pricing equation 2.6:

$$q(z_j; k', b', c') = \frac{1}{1 + r_f(1 - \tau)} \left[(1 - g P_{\text{def}}(z_j; k', b', c')) + g P_{\text{def}}(z_j; k', b', c') \frac{R(k', c')}{b'k'} \right].$$

When $g = 0$ (recovery covers the debt, including the no-debt case $b' = 0$) the bond is risk-free, $q = 1/(1 + r_f(1 - \tau))$.

Bellman right-hand side. For state (z_j, k_i, b_m) and action (k', b', c') ,

$$\text{RHS}(z_j, k_i, b_m; k', b', c') = D(z_j, k_i, b_m, i, b', c') + \frac{1}{1 + r_f} C(z_j; k', b', c'), \quad i = \frac{k'}{k_i} - (1 - \delta),$$

where D is the two-subperiod net payout 2.9, evaluated with the bond price $q(z_j; k', b', c')$ above and discount factor $\beta = 1/(1 + r_f)$. This is exactly the Bellman objective of the optimal value equation 2.11, restricted to the action grid.

3.1.3 Algorithm

Step 0: Initialization. Set the round counter $n = 0$. Initialize the value to zero on every grid point, $V^{(0)}(z_j, k_i, b_m) = 0$. No separate price or policy guess is needed: because $V^{(0)} \equiv 0$ gives $\hat{V} \equiv 0$ and hence $P_{\text{def}} \equiv 0$, the implied initial bond price is the risk-free $q^{(0)} = 1/(1 + r_f(1 - \tau))$ at every action, and the policy is defined by the first improvement step below.

Step 1: Debt price at the current value. For every (z_j, k', b', c') on the action grid, recompute C , P_{def} , and $q^{(n)}$ from $V^{(n)}$ using the formulas above.

Step 2: Policy improvement (greedy). For each state (z_j, k_i, b_m) , evaluate RHS over the *entire* action grid and keep the maximizer:

$$h^{(n+1)}(z_j, k_i, b_m) = \arg \max_{(k', b', c') \in \mathcal{A}} \text{RHS}(z_j, k_i, b_m; k', b', c').$$

Step 3: Policy evaluation (exact). Holding the policy $h^{(n+1)}$ and the bond price $q^{(n)}$ fixed, solve for the value of following $h^{(n+1)}$ forever. For each state $s = (z_j, k_i, b_m)$ with chosen action $(k'(s), b'(s), c'(s))$ this is the linear fixed point

$$V(s) = D(s, h^{(n+1)}(s), q^{(n)}) + \frac{1}{1 + r_f} \sum_{l=1}^{N_z} P_{jl} \max\{\hat{V}(z_l, k'(s), b'(s) - c'(s)), 0\}.$$

The only nonlinearity in this fixed point is the limited-liability kink $\max\{\hat{V}, 0\}$. Fixing the *solvent set* of next-period nodes with $\hat{V} > 0$ removes the kink and makes the right-hand side linear in V , so stacking all $S = N_z N_k N_b$ states gives the linear system $(I - \beta A) V = D$ where $\beta = 1/(1 + r_f)$, I is the $S \times S$ identity, D is the (fixed- q) dividend vector, and A is the transition matrix (the bilinear interpolation weights times the Tauchen probabilities, zeroed on defaulting next states). This system is solved directly for $V^{(n+1)}$ with `tf.linalg.solve(I -  $\beta A$ , D)`, then update the solvent set and re-solve until it stops changing. See section 3.1.5 for details.

Step 4: Convergence. Stop when both the value and the policy have stabilized:

$$\|V^{(n+1)} - V^{(n)}\|_{\infty} < \epsilon \quad \text{and} \quad h^{(n+1)} = h^{(n)} \text{ (identical maximizers on the grid).}$$

Otherwise set $n \leftarrow n + 1$ and return to Step 1.

Output. The converged triple is returned as:

$$V, \quad h = (i, k', b', c'), \quad q(z, k', b', c')$$

where the equity value reported to shareholders is $\max\{V, 0\}$ and the raw value $V < 0$ flags the default states. Default convergence settings are $\epsilon = 10^{-8}$ with a few hundred rounds.

3.1.4 Correctness of the solution

The model solution is a coupled fixed point: the value V sets the default probability and hence the price q , while q enters the payout D and determines V . As flagged in Proposition 2.1, the map $V \mapsto T(V, q(V))$ is not obviously a contraction, so we do not try to prove that the algorithm converges from any starting point. What we can prove is that each policy-evaluation step is exact, and that any point the algorithm stops at is a genuine model solution consistent with Proposition 2.1.

Proposition 3.1: Grid correctness of Howard policy iteration

On the discrete state-action grid, with discount $\beta = 1/(1 + r_f) < 1$:

1. **Policy evaluation is exact.** Fix a price schedule q and a policy h . The evaluation step (Step 3) computes an exact and unique solution.
2. **Correctness at the converged solution.** If the iteration meets the Step 4 convergence rule, then the returned triple (V, h, q) jointly satisfies the discrete default rule 2.4, the bond-pricing break-even condition 2.6, and the Bellman optimality equation 2.11 at once. It is therefore a model solution on the grid as defined in Proposition 2.1.

Proof of Statement 1. With q and h fixed, the payout $D(s)$ at each state s is a fixed number, and Step 3 looks for a value array V solving

$$(\mathcal{T}V)(s) = D(s) + \frac{1}{1 + r_f} \sum_l P_{jl} \max\{\widehat{V}(z_l, k', b' - c'), 0\}, \quad V = \mathcal{T}V.$$

Take any two value arrays V and V' . Three operations separate them inside $\mathcal{T}$, and none can amplify a difference: (i) the interpolated value $\widehat{V}_l$ is a weighted average of entries of V with non-negative weights that sum to one, so $|\widehat{V}_l - \widehat{V}'_l| \leq \|V - V'\|_\infty$; (ii) the floor $\max\{\cdot, 0\}$ moves its argument by at most the change in that argument, so it cannot increase the gap; (iii) the row $P_{j\cdot}$ is another non-negative average that sums to one. Hence at every state:

$$\|\mathcal{T}V - \mathcal{T}V'\|_\infty \leq \beta \|V - V'\|_\infty, \quad \beta = \frac{1}{1 + r_f} < 1.$$

A map that shrinks all distances by the fixed factor $\beta < 1$ can have only one fixed point, and iterating $\mathcal{T}$ from any start reaches it. This unique V is what Step 3 returns.

Proof of Statement 2. Suppose at round n the Step 4 test passes, so the policy and value no longer move: $h^{(n+1)} = h^{(n)} =: h$ and $V^{(n+1)} = V^{(n)} =: V$; write $q := q^{(n)}$. Read off the three conditions directly from the steps that produced them.

- *Bond pricing.* Step 1 built q from V by counting, for each action, the next-period states where $V < 0$ and plugging that default probability into the break-even formula 2.6. So q already satisfies bond pricing at the default states implied by V .
- *Policy optimality.* Step 2 set h to the action that maximizes the Bellman right-hand side under q . Because h did not change this round, h is the greedy maximizer for the current (V, q) .
- *Bellman equation.* Step 3 set V to the exact value of following h under q (Part 1). Because V did not change, it equals that on-policy value; and since h is the greedy maximizer, that value also equals the maximum of the right-hand side over all actions, which is exactly the Bellman optimality equation 2.11.

All three conditions of Proposition 2.1 hold at (V, h, q) . $\square$

What is not claimed. The proposition says nothing about global convergence or the uniqueness of the converged solution. The reason is that the coupled map is not a contraction: the default probability counts the states where $V < 0$, so it is a step function of V , and a tiny change in V that pushes one state across zero makes q jump by a finite amount. Two consequences follow, and we do not rule either out. (1) The solution need not be unique: several triples (V, h, q) can all pass the Step 4 test. (2) Which one the algorithm reaches can depend on the initialization, so this is not a global convergence guarantee. In practice, from the cold start $V^{(0)} = 0$ the iteration settles to a particular coupled fixed point, but that is an empirical observation, not a theorem.

What it means for model solver validation. If we use this algorithm as the benchmark solver, apply the same set of bounds (parameter values) and initialization, then it will converge to a solution triplet (V, h, q) that jointly satisfies the Bellman equation, the policy-maximization condition, and the bond pricing equation, which defines the solution of the risky debt model (Proposition 2.1).

Any neural networks that approximate the value and policy function should be evaluated on-grid and against this benchmark. Deviation must flag a serious issue with neural solvers.

3.1.5 Implementation notes

Interpolation of next-period value. Since we use finer action grids $(\log k', b' - c')$ than the state grids $(\log k, b - c)$, the optimal action can land between the state grid points, thus the next-period equity value $\hat{V}(\log k', b' - c')$ needs to be interpolated because the current $\hat{V}$ is evaluated on state grids $(\log k, b - c)$.

To interpolate the next-period value, for a given $(\log k', b' - c')$ lands within a grid cell bounded by its four nearest neighbors $Q_{11} = (\log k_1, b_1)$, $Q_{21} = (\log k_2, b_1)$, $Q_{12} = (\log k_1, b_2)$, and $Q_{22} = (\log k_2, b_2)$, we perform successive linear interpolations along each axis. First, we interpolate linearly along the $\log k$ -axis between the bottom pairs and top pairs of points to find the intermediate values $R_1(\log k, b_1)$ and $R_2(\log k, b_2)$, using the formula

$$R_i = \frac{\log k_2 - \log k}{\log k_2 - \log k_1} \hat{V}(z_l; \log k_1, b_i) + \frac{\log k - \log k_1}{\log k_2 - \log k_1} \hat{V}(z_l; \log k_2, b_i)$$

for $i \in \{1, 2\}$. Next, we interpolate linearly along the b -axis between these two intermediate results to obtain the final value, yielding

$$\hat{V}(z_l; \log k', b' - c') = \frac{b_2 - b}{b_2 - b_1} R_1 + \frac{b - b_1}{b_2 - b_1} R_2.$$

Combining these steps into a single expression clarifies that the weight of each corner Q_{ij} is proportional to the area of the opposite rectangle created by the point $(\log k', b' - c')$ within the cell.

Exact policy evaluation. Step 3 solves for the greedy policy $h^{(n+1)}$ from Step 2 as follows. Stack the $S = N_z N_k N_b$ states into $V \in \mathbb{R}^S$, $s = (z_j, k_i, b_m)$, with the chosen action $h^{(n+1)}(s) = (k'(s), b'(s), c'(s))$ and the round's bond price $q^{(n)}$ held fixed. The dividend vector $D \in \mathbb{R}^S$ is constant. Write $\hat{V}_{s,l} \equiv \hat{V}(z_l, k'(s), b'(s) - c'(s))$ for the interpolated next-period value. Now with the solvent set $\{(s, l) : \hat{V}_{s,l} > 0\}$ held fixed, $\max\{\hat{V}_{s,l}, 0\} = \hat{V}_{s,l} \mathbf{1}\{\hat{V}_{s,l} > 0\}$ is linear and the Step 3 right-hand side is a linear system $D + \beta AV$. Here the A matrix is constructed as

$$A_{s, (z_l, Q_{ij}(s))} = P_{jl} \mathbf{1}\{\hat{V}_{s,l} > 0\} w_{ij}(s),$$

where $Q_{ij}(s)$ are the four cell corners that bracket $(\log k'(s), b'(s) - c'(s))$ and $w_{ij}(s)$ are their bilinear weights from the interpolation ($\sum_{ij} w_{ij} = 1$). Every other defaulting entry is zero.

To solve for V in the linear system $V = D + \beta AV$, we use an active-set (semismooth-Newton) iteration on the solvent set: initialize it from the entry value, then (i) assemble $M = I - \beta A$ with the current set, I the $S \times S$ identity, and solve the dense system $M V^+ = D$ directly; (ii) recompute the set from V^+ . Repeat until the set stops changing or the relative update drops below 10^{-10} , capped at 50 inner steps. The matrix A is assembled sparsely into a dense array and the solve is a double-precision LU through `tf.linalg.solve`:

```

active = (Vhat(V) > 0)                                # solvent set from entry V; Vhat = V
               interpolated at (k', b'-c')
for _ in range(max_iter):                             # inner active-set loop, within Step 3
    M = tf.eye(S) - beta * A(active)                  # A = Tauchen P x bilinear weights, masked
               by active
    V_new = tf.linalg.solve(M, D)                     # exact dense LU solve at the fixed solvent
               set
    active_new = (Vhat(V_new) > 0)
    stable = tf.reduce_all(active_new == active)
    converged = rel_change(V_new, V) < 1e-10          # max|V_new - V| / (1 + max|V|)
    V, active = V_new, active_new
    if stable or converged:
        break

```

As a regression check, a separate unit test re-solves the same model with the slower plain value iteration (a single Bellman backup per sweep, no inner linear solve) and confirms it reaches the same fixed point as this Howard solver. Because that path never assembles A nor calls the linear solve, the agreement is independent evidence that the active-set assembly and the dense solve are correct.

Joint value-debt price update. The endogenous price $q^{(n)}$ is recomputed from the last value $V^{(n)}$ at the start of every round (Step 1), and the value, policy, and price reach a joint fixed point together. This differs from a nested algorithm in [Strebulaev and Whited \(2012, Section 3.6\)](#) that would freeze the bond price across a full inner value-iteration solve.

Compute cost. The main cost is the greedy search in Step 2: for each of the $N_z N_k N_b$ states it scans all $N_{k'} N_{b'} N_{c'}$ actions, so one improvement is $O(N_z N_k N_b \cdot N_{k'} N_{b'} N_{c'})$, and the bond-price array $q(z_j; k', b', c')$ spans the full state-shock by action grid. This curse of dimensionality is exactly why this grid solver serves only as a benchmark, not as the estimation-loop solver.

Core round vectorized. The continuation, default probability, and price depend on the current shock z and the action, but *not* on the current (k, b) state; only the payout D does. Computing them once per round over the action grid (shape $[N_z, |\mathcal{A}|]$) and then broadcasting to states by the current shock index is both the speed-up and the easiest place to introduce a bug. The code is:

```

bpp = bp - cp                                         # next net debt b' - c'
vprime = interp_grid(V, logk, b_grid, kp, bpp)       # V(z', k', b'-c')
cont = P @ tf.maximum(vprime, 0.0)                  # E[ max(V', 0) | z ]
pdef = P @ tf.cast(vprime < 0.0, f64)               # P_def = E[ 1{V'<0} | z ]
q      = bond_price(pdef, kp, cp, bp, chi, delta, bond_disc)

i_rate = kp / k - (1 - delta)                       # i implied by the chosen k'
D       = dividend(z, k, b, i_rate, bp, cp, q, smooth=False).D # exact D
rhs     = D + cont / (1 + rf)                       # then argmax over actions per state

```

The action grid is flattened to one axis and the inner search is chunked over it for memory only; chunking does not change the result.

Choices the algorithm leaves implicit. Three are essential and silently wrong if missed.

- *Interpolate at the net debt $b' - c'$, not at b' .* Both the continuation and P_{def} query $\hat{V}$ at $(k', b' - c')$ with the *same* interpolation. Using b' , or two different interpolations, breaks the consistency between pricing and continuation.
- *Keep V unclamped between rounds.* The array fed back into `interp_grid` is the raw V , which may be negative: $V < 0$ is exactly how default is detected. If the iterate is stored as $\max\{V, 0\}$, then $P_{\text{def}} \equiv 0$, every bond is risk-free, and default risk is silently lost. Only the final *reported* shareholder value is $\max\{V, 0\}$.
- *Default is read off the interpolated $\hat{V}$,* the same off-grid quantity used for the continuation, not off the grid-node values of V .

TensorFlow specifics that change the result. A few choices are load-bearing for reproducing the same numbers on the same hardware.

- *float64 on CPU.* Every grid, the Tauchen matrix P , and the value array are `float64`; on Apple Silicon there is no float64 GPU kernel, so the solve runs on CPU. A float32 path moves the converged value well above the $\epsilon = 10^{-8}$ tolerance.
- *Row convention of the expectation.* $P_{jl} = \Pr(z_l | z_j)$, so the expectation is the single matmul `P @ vprime` with row j the current shock; transposing P is a silent error.
- *Deterministic `argmax`.* `tf.argmax` returns the *first* maximizer. This stable tie-break is what lets the “policy unchanged” half of the Step 4 test ever fire.
- *Exact (non-smoothed) dividend.* The benchmark calls `dividend(..., smooth=False)`: the hard indicators of the payout, not the smoothed approximation used in training.
- *Price frozen inside Step 3.* The exact evaluation solves $(I - \beta A)V = D$ with q (hence D) held at the round’s entry value and the solvent set $A = \mathbf{1}\{\hat{V} > 0\}$ refreshed between linear solves; recomputing q inside the solve would make it nonlinear and lose exactness.

3.2 Neural policy iteration

This is the project’s *default* model solver. Instead of solving the model on a grid for one parameter vector at a time, it trains neural networks that approximate the universal value and policy functions over the *joint state and parameter space* $(z, k, b) \times \Theta$, so one trained model returns (V, h, q) for any parameter vector in the estimation range. The algorithm is adopted from the “Block 1” policy iteration of Duarte and Fonseca (2026, Sec. 3.1). It alternates between two steps: first evaluate the value of the current policy, then improve the policy, both through stochastic gradient descent on sampled states and auto-differentiation. The circularity between the value and the bond price is broken with a slowly-updated *target* copy of the value network.

3.2.1 Neural networks

Let Θ be the parameter vector to be estimated. We use four networks with the same architecture, each with its own weights denoted by ϕ :

- One **value network** $V_\phi(z, k, b; \Theta)$ returning a scalar equity value (no squashing);
- Three **policy networks** $h_\phi^i, h_\phi^b, h_\phi^c$, one per action (i, b', c') .

Normalized inputs. Every network sees the state and the parameters rescaled to $[-1, 1]$ by

$$\tilde{x} = \frac{2(x - x_{\min})}{x_{\max} - x_{\min}} - 1.$$

The state box is the same parameter-dependent box as the grid bounds of the previous section: $\log z$ over its ± 2.5 unconditional standard deviations, $k \in [\underline{k}, \bar{k}]$, and $b \in [\underline{b}, \bar{b}]$. Because the box depends on Θ , the same raw state maps to different normalized inputs across parameter vectors, and the parameter bounds $[x_{\min}, x_{\max}]$ are the estimation range specified by user.

Shared trunk network architecture. All four networks share one feed-forward architecture, the *trunk*, which takes the normalized state only. The parameters do not enter the trunk directly but are attached through the FiLM generators below. The trunk has $L = 3$ hidden layers of $U = 128$ units each. Writing h_0 for the normalized state input, a plain unmodulated layer is

$$h_{l+1} = \varphi(W_l h_l + c_l), \quad l = 0, \dots, L-1, \quad (3.3)$$

with network weights W_l , bias c_l , and SiLU activation $\varphi(x) = x \sigma(x)$ where σ is the logistic sigmoid. SiLU is chosen because it is smooth with a nonzero gradient almost everywhere.

The final *output head* is a single linear layer $U \rightarrow 1$ that outputs one raw scalar and with no activation, so its range is unrestricted. It does not by itself enforce any bound. The value network takes that raw scalar directly as V_ϕ (equity value is unbounded and can be negative). Each policy network instead maps its raw scalar into the action’s feasible interval as described below.

Feasible policy outputs. Each policy network’s one raw scalar output x is squashed into its feasible interval by a biased sigmoid,

$$a = a_{\min} + (a_{\max} - a_{\min}) \sigma(x + x_0),$$

so a stays strictly inside $[a_{\min}, a_{\max}]$. The bounds are those of the action space (i, b', c') : the investment-rate bounds depend on k such that $k' = (1 + i - \delta)k$ stays in $[\underline{k}, \bar{k}]$, while $b' \in [0, \bar{b}']$ and

$c' \in [0, \bar{c}']$. The gross-debt network uses $x_0 = -4$, so $\sigma(-4) \approx 0.018$ starts the debt policy low and stabilizes early training before the bond price is learned; the other two actions use $x_0 = 0$. Because the squashing function is a sigmoid, an action pushed close to either end of its feasible interval saturates and the gradient reaching the policy there becomes very small; with the bias $x_0 = -4$ the gross-debt policy in particular starts in this flat region, which steadies early training but also means it can move off its low initial debt only slowly, until the raw output climbs out of the saturated range.

FiLM parameter conditioning. Learning the universal policy and value function over a high-dimensional state-by-parameter space requires a lot of network capacity. To reduce this burden, we do not use the concatenated (state, parameters) vector as the network input. Instead, we use a set of small auxiliary networks to learn the mappings from the model parameters Θ to two pre-activation parameters, (γ_l, δ_l) , for each main network layer l . These activation parameters then moderate the main networks, which map states (z, k, b) to actions (i, b', c') and value V .

This is known as feature-wise linear modulation (FiLM) from [Perez et al. \(2018\)](#): the main networks (“trunks”) learn the state-dependence as usual, while the parameter-conditioning is injected layer by layer through a scale and a shift (γ_l, δ_l) . Each of the small auxiliary network is called a FiLM generator and is attached to the four trunk networks. Concretely, there is one FiLM generator per (trunk, layer) pair, hence $4 \times L = 12$ generators with independent weights. Index a generator $G_l^j(\Theta)$ by layer l of trunk network $j \in \{V, h^i, h^b, h^c\}$. All generators consume the same normalized parameter batch Θ and have the same architecture: one hidden layer of 32 units with SiLU activation. For a mini-batch of N parameter vectors, $G_l^j(\Theta)$ returns $(\gamma_l^j, \delta_l^j) \in \mathbb{R}^{N \times U}$ for each trunk j .

The generators’ output (γ_l, δ_l) then modulate the trunk’s pre-activation, replacing the plain layer in equation 3.3 with

$$h_{l+1} = \varphi(\gamma_l \odot (W_l h_l + c_l) + \delta_l), \quad l = 0, \dots, L-1, \quad (3.4)$$

where $h_l \in \mathbb{R}^U$ is the layer- l activation, with h_0 the normalized state; W_l and c_l are the trunk weight matrix and bias; $\gamma_l, \delta_l \in \mathbb{R}^U$ are the generator’s scale and shift; $\odot$ is elementwise multiplication; and φ is the SiLU activation. The generators are identity-initialized ($\gamma_l = \mathbf{1}$, $\delta_l = \mathbf{0}$ at the start), so each network begins as its plain state trunk and learns parameter-dependence as a departure from the identity as training proceeds. Figure 3.1 summarizes the full architecture with the trunk networks and their FiLM generator networks.

3.2.2 Building blocks evaluated by sampling

At a sampled state (z, k, b) and parameters Θ , the policy networks give controls $i = h_\phi^i$, $b' = h_\phi^b$, $c' = h_\phi^c$, hence $k' = (1 + i - \delta)k$ and next net debt $b' - c'$. The Bellman right-hand side is assembled from the same economic objects as the benchmark, but everything that must be differentiated is made smooth.

Conditional expectation (Gauss-Hermite quadrature). The expectation over next-period productivity is an integral across the stochastic shock $\varepsilon \sim \mathcal{N}(0, 1)$ underlying $\log z' = \rho \log z + \sigma \varepsilon$.

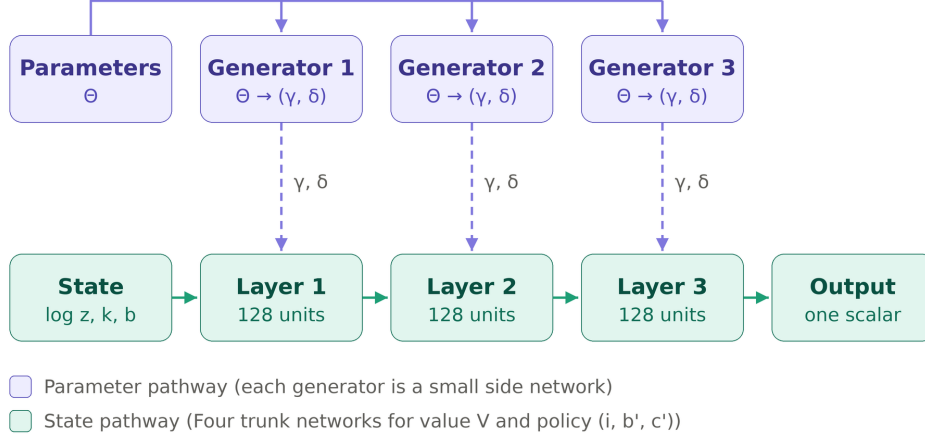


Figure 3.1: FiLM data flow: the normalized state enters the trunk, while the normalized parameters drive the per-layer generators that produce the scale γ_l and shift δ_l modulating each trunk layer.

Gauss-Hermite quadrature numerically computes this integral with Q nodes $\{x_q\}$ and weights $\{w_q\}$,

$$\begin{aligned}
 \mathbb{E}_{z'|z}[f(z')] &= \int_{-\infty}^{\infty} f(e^{\rho \log z + \sigma \varepsilon}) \frac{1}{\sqrt{2\pi}} e^{-\varepsilon^2/2} d\varepsilon \\
 &\approx \frac{1}{\sqrt{\pi}} \sum_{q=1}^Q w_q f(e^{\rho \log z + \sigma \sqrt{2} x_q})
 \end{aligned} \tag{3.5}$$

The expected continuation value $C = \mathbb{E}_{z'|z}[\max\{V_\phi(\log z', k', b' - c'; \Theta), 0\}]$ is computed using this quadrature in equation 3.5 and the *current* value network V_ϕ :

$$C \approx \frac{1}{\sqrt{\pi}} \sum_{q=1}^Q w_q \max\{V_\phi(\rho \log z + \sigma \sqrt{2} x_q, k', b' - c'; \Theta), 0\}.$$

Given fixed Q , the nodes x_q and weights w_q are constants computed once using standard Golub-Welsch eigenvalue method with `numpy.polynomial.hermite.hermgauss`, and then frozen as a `tf.constant`. The node evaluation and weighted sum then run natively in TensorFlow inside the gradient graph.

Approximation bias. The integrand $\max\{V_\phi(z'(\varepsilon), \cdot), 0\}$ is smooth in ε except for a single kink at the default boundary $\{\varepsilon^* : V_\phi(z'(\varepsilon^*), \cdot) = 0\}$. Gauss-Hermite implicitly integrates a smooth polynomial fit, which cannot represent that kink, so the approximation bias is confined to the default region and concentrated at its boundary around ε^* . Its size is governed by the shock mass there: when the firm is safe the boundary ε^* lies far in the tail of $\Phi(\varepsilon)$, the shock mass is negligible, and the quadrature is almost exact; as default risk rises the boundary moves toward the mode and the bias grows.

We address this in two ways. First, we report a diagnostic metric: the gap between the quadrature continuation and the *exact* expected value evaluated on the Tauchen grid from equation 3.1. Second, we consider replacing the hard floor with the smooth gate $\max\{V', 0\} \approx \kappa \log(1 + \exp(V'/\kappa))$, which restores the accuracy of the quadrature across the entire ε range but loses the enforcement of exact limited liability. In practice, the diagnostic will tell which one is better.

Default probability (Taylor expansion). The bond price needs $P_{\text{def}} = \Pr(V' < 0)$. A second quadrature would be costly, so it is obtained in closed form. Write the continuation value as a function of the shock, $V'(\varepsilon) = V_{\text{target}}(z'(\varepsilon), k', b' - c'; \Theta)$ with $z'(\varepsilon) = e^{\rho \log z + \sigma \varepsilon}$, and expand to second order around $\varepsilon = 0$,

$$V'(\varepsilon) \approx V_0 + a\varepsilon + \frac{1}{2}c\varepsilon^2, \quad a = \left. \frac{dV'}{d\varepsilon} \right|_0, \quad c = \left. \frac{d^2V'}{d\varepsilon^2} \right|_0, \quad (3.6)$$

with a, c from automatic differentiation of the *target* network and $V_0 = V(z'(0), \cdot)$. Using the *target* network is key to break the circularity of value and bond pricing via default probability.

How this works? The default probability is, exactly, the standard-normal measure of the shocks at which the continuation value is negative,

$$P_{\text{def}} = \Pr(V'(\varepsilon) < 0) = \int_{\{\varepsilon : V'(\varepsilon) < 0\}} \phi(\varepsilon) d\varepsilon, \quad \phi(\varepsilon) = \frac{1}{\sqrt{2\pi}} e^{-\varepsilon^2/2},$$

with $\varepsilon \sim \mathcal{N}(0, 1)$ and $(k', b' - c')$ fixed by the current-period action, so ε is the only randomness. Substitute the quadratic approximation $\tilde{V}(\varepsilon) = V_0 + a\varepsilon + \frac{1}{2}c\varepsilon^2$ for V' : the default set becomes $\{\varepsilon : \tilde{V}(\varepsilon) < 0\}$, and because a parabola changes sign at most twice this set is a union of at most two intervals whose endpoints are the roots

$$\varepsilon_{\pm} = \frac{-a \pm \sqrt{\Delta}}{c}, \quad \Delta = a^2 - 2cV_0.$$

Writing $\varepsilon_{\text{lo}} \leq \varepsilon_{\text{hi}}$ for the two roots sorted, the default probability follows in closed form from the standard-normal CDF Φ , by case on the curvature c and the discriminant Δ :

$$P_{\text{def}} = \begin{cases} \Phi(\varepsilon_{\text{hi}}) - \Phi(\varepsilon_{\text{lo}}), & c > 0, \Delta > 0, \\ 0, & c > 0, \Delta \leq 0, \\ \Phi(\varepsilon_{\text{lo}}) + 1 - \Phi(\varepsilon_{\text{hi}}), & c < 0, \Delta > 0, \\ 1, & c < 0, \Delta \leq 0, \\ \Phi(-V_0/a), & c \approx 0, a > 0, \\ 1 - \Phi(-V_0/a), & c \approx 0, a < 0, \\ \mathbf{1}\{V_0 < 0\}, & c \approx 0, a \approx 0. \end{cases} \quad (3.7)$$

Shape of the parabola. The regime switches in equation 3.7 depend on the geometry of the quadratic function $\tilde{V}$, which is pinned down by c and a from auto-differentiation of the target value network. Default probability is the fraction where $\{\varepsilon : \tilde{V}(\varepsilon) < 0\}$. When $c > 0$ and $\Delta > 0$, $\tilde{V}$ is U-shaped and below zero between $[\varepsilon_{\text{lo}}, \varepsilon_{\text{hi}}]$. When $c > 0$ and $\Delta \leq 0$, $\tilde{V}$ is U-shaped but always above zero, so default probability is always zero. When $c < 0$ and $\Delta > 0$, $\tilde{V}$ is inverse U-shaped and the default set is the union $\{\varepsilon < \varepsilon_{\text{lo}}\} \cup \{\varepsilon > \varepsilon_{\text{hi}}\}$. When $c < 0$ and $\Delta < 0$, $\tilde{V}$ is always below zero and the default probability is one. Finally, when $c \approx 0$, $\tilde{V} \approx V_0 + a\varepsilon$ is linear and crosses zero once at $-V_0/a$.

Approximation errors. There are two layers of potential approximation errors: (i) error in the learned value network V' which is auto-differentiated to obtain a and c ; (ii) the truncation of V' with quadratic Taylor expansion. Beyond these two, the formula above is exact and is the true Gaussian measure of $\{\tilde{V} < 0\}$ evaluated analytically rather than sampled, so it adds no Monte-Carlo

or quadrature integration error. The value network error is non-avoidable for any method that needs to approximate a value function with no closed-form.

The Taylor truncation error of V' is governed by the third- and higher-order curvature of $V'(\varepsilon)$ over the range of ε . That curvature is *not* guaranteed to be negligible: it is an empirical property of the learned value surface and is sensitive to modeling choices. A larger σ can stretch the shock across a more nonlinear part of V' . Thus, the truncation error should be treated as a quantity to be *measured*, not assumed away. It is directly observable on discrete grids, because the exact default probability $\mathbb{E}_{z'|z}[\mathbf{1}\{V' < 0\}]$ can be computed as the probability-weighted count of defaulting next-period nodes on the Tauchen grid (exactly what the Howard's policy iteration algorithm does). The Taylor error is then the gap between P_{def} and the on-grid reference probability. This is reported as a diagnostic.

Why this is useful? The main benefit of this quadratic approximation is differentiability. The hard indicator $\mathbf{1}\{V' < 0\}$ has zero gradient almost everywhere, so a policy trained against it would receive no signal about how its choices move the default probability. The analytic P_{def} , by contrast, is a smooth function of (V_0, a, c) , hence of the controls and the parameters, so the policy can be trained against the price impact of its own default risk: a riskier (k', b', c') raises P_{def} , lowers the bond price q , and the gradient carries that penalty back to the policy networks. Compared with the All-in-One unbiased estimator from [Maliar et al. \(2021\)](#), this method is deterministic, easy to implement and debug, and avoids potential errors with multiple Monte-Carlo draws. Compared with [Wong \(2026\)](#)'s approach that trains a separate price network, this approach reduces the number of training objectives and the potential neural approximation error.

Bond price. Given P_{def} , the bond price is directly computed as,

$$q = \frac{1}{1 + r_f(1 - \tau)} \left[(1 - g P_{\text{def}}) + g P_{\text{def}} \frac{R(k', c')}{b'k'} \right], \quad (3.8)$$

with deterministic gate $g = \mathbf{1}\{R < b'k'\}$ and recovery $R(k', c') = c'k' + \chi(1 - \delta)k'$.

Net payout with smooth approximation. The net payout $D(z, k, b, i, b', c', q)$ is evaluated with this q . Recall from equation 2.9:

$$D = d_2 + \min\{d_1, 0\} - \eta(d_1) \cdot \mathbf{1}\{d_1 < 0\} - \eta(d_2) \cdot \mathbf{1}\{d_2 < 0\},$$

where equity issuance cost is $\eta(d) = \lambda_0 k + \lambda_1 |d|$, pre-production net cash flow is $d_1 = q \cdot b' \cdot k' - b \cdot k - c_f$, and post-production cash flow is $z A_\pi k^\xi + \max\{d_1, 0\} - i \cdot k - c'k' - \psi(I, k)$ with adjustment cost $\psi(I, k) = \frac{1}{2} \gamma_1 \frac{I^2}{k} + \gamma_0 k \mathbf{1}\{I > 0\}$.

For gradient-based training, the non-differentiable parts of D are approximated with smooth functions:

$$\begin{aligned} \mathbf{1}\{x < 0\} &\approx \frac{1}{1 + e^{x/\kappa}}, \\ \mathbf{1}\{x > 0\} &\approx \frac{e^{x/\kappa}}{1 + e^{x/\kappa}}, \\ \max\{x, 0\} &\approx \kappa \log(1 + e^{x/\kappa}), \\ \min\{x, 0\} &\approx -\kappa \log(1 + e^{-x/\kappa}), \end{aligned} \quad (3.9)$$

where each smooth surrogate converges to the hard indicator or kink as the hyperparameter $\kappa \rightarrow 0$. The absolute function $|x|$ in issuance cost $\eta(x)$ can be rewritten as $\max\{x, 0\} - \min\{x, 0\}$, and is also replaced with the smooth surrogates. The default temperature is set to $\kappa = 10^{-3}$.

Bellman. Now we have everything needed to compute both sides of the Bellman equation:

$$V_\phi(z, k, b; \Theta) = D(z, b, k, \hat{i}, \hat{b}', \hat{k}', q) + \frac{1}{1 + r_f} \mathbb{E}_{z'|z} [\max\{V_\phi(z', \hat{k}', \hat{b}' - \hat{c}'; \Theta), 0\}] \quad (3.10)$$

where the ‘hat’ denotes policy network output, for example, $\hat{i} = h_\phi^i(z, b, k; \Theta)$. Three separate policy networks generate $(\hat{i}, \hat{b}', \hat{c}')$, and it follows that $\hat{k}' = (1 + \hat{i} - \delta)k$. Crucially, the bond price $q(\tilde{V}_\phi^-)$ is computed from analytical default probability P_{def} from equation 3.7, which uses a quadratic Taylor expansion to approximate a slowly-updated *target* value network V_ϕ^- around zero. This breaks the circularity between bond price q and *current* value network, V_ϕ , on both sides.

3.2.3 Algorithm

Step 0: Initialization. Set stateless master seeds. Build the value network, the three policy networks, and a *target* value network (a frozen copy of V_ϕ used only for the bond price). All FiLM generators are identity-initialized; the gross-debt policy is biased low ($x_0 = -4$).

Step 1: Sample a mini-batch. Draw i.i.d. $N = 8,192$ rows of $(z, k, b; \Theta)$: first sample uniformly the parameter vector Θ on the current bounds. Then for each Θ , sample uniformly the state vector (z, k, b) on that parameter’s implied state box. Uniform sampling the entire box, rather than a simulated ergodic set, is what makes the trained networks universal over the parameter range.

Step 2: Policy evaluation. Fix the policy networks $(h_\phi^i, h_\phi^b, h_\phi^c)$, take one Adam step on the value weights ϕ_v to reduce the mean squared Bellman residual

$$\mathcal{L}_V = \frac{1}{N} \sum_{i=1}^N \left(V_{\phi_v} - D - \frac{1}{1 + r_f} \mathbb{E}_{z'|z} [\max\{V'_{\phi_v}, 0\}] \right)_i^2, \quad (3.11)$$

$$\phi_v \leftarrow \phi_v - \alpha \nabla_{\phi} \mathcal{L}_V, \quad (3.12)$$

where the residual is defined explicitly in equation 3.10. In this step, gradient flows through current value network V_{ϕ_v} . Payout $D(z, b, k, \hat{i}, \hat{b}', \hat{k}', q(V^-))$ is fixed (policy weights frozen) and bond price q use the frozen target value V^- to analytically compute default probability from equation 3.7. Expectation $\mathbb{E}[f(z')|z]$ is computed using Gauss quadrature from equation 3.5. Kinks and hard indicators in D and the min/max operators are replaced by smooth surrogates in equation 3.9.

The value weights ϕ_v include the value network’s trunk weights (W_l, c_l) and its per-layer FiLM generator weights (γ_l, δ_l) (see equation 3.4). One Adam step jointly updates them all.

Step 3: Policy improvement. Fix the value weights ϕ_v and using the current V_{ϕ_v} , take one Adam step on the policy weights ϕ_h to maximize the mean Bellman right-hand side,

$$\mathcal{L}_h = -\frac{1}{N} \sum_{i=1}^N \left(D(z, b, k, \hat{i}, \hat{b}', \hat{k}', q) + \frac{1}{1 + r_f} \mathbb{E}_{z'|z} [\max\{V_{\phi_v}, 0\}] \right)_i, \quad (3.13)$$

$$\phi_h \leftarrow \phi_h - \alpha \nabla_{\phi} \mathcal{L}_h, \quad (3.14)$$

where the gradient flows through the three policy networks’ implied actions $\hat{i}, \hat{b}', \hat{c}'$ into both the payout D and the expected equity value $V_\phi(z', \hat{k}', \hat{b}' - \hat{c}')$ averaged across z' . Importantly, the policy gradient also flows into the bond price q , so the firm internalizes the price impact of its own default risk. To see this, the actions $(\hat{k}', \hat{b}', \hat{c}')$ directly enter the bond price q ’s formula through recovery and

default gates (equation 3.8), but also indirectly enter the default probability formula (equation 3.7) via the target value network V_ϕ^- evaluated at these actions.

The policy weights ϕ_h include the policy network’s trunk weights (W_l, c_l) and its per-layer FiLM generator weights (γ_l, δ_l) (see equation 3.4). One Adam step jointly updates them all.

Step 4: Epochs and schedule. Repeat steps 1–3 for 500 gradient steps per epoch. At the start of every epoch the target network is reset to the current value weights, and the shared learning rate decays by 1% (floored at 10^{-6}). Training runs for a fixed number of epoches.¹

Output. The final output returns the trained value and policy networks $(V_\phi, h_\phi^i, h_\phi^b, h_\phi^c)$ defined over the whole state×parameter space. Default probability $P(z, k', b' - c')$ and bond price $q(z, k', b' - c')$ are then computed analytically from equation 3.7 and 3.8.

3.2.4 Correctness of the solution

NPI outputs reflect a local optimum by approximation. There is no mathematical guarantee that the neural policy iteration converges to an exact and unique global optimum.

- Trained networks are smooth approximation to the true value and policy functions.
- Stochastic gradient descent with i.i.d. sampling is an unbiased optimization procedure.
- Bellman right-hand-side is non-smooth and the policy improvement step may converge to one of the many local optimums.

Thus, the output solution depends on the network approximation error, the optimization process (e.g., training budget), and initialization and sampling choice.

Because there is no fixed-point guarantee, the outputs must first be validated on grid against the benchmark solver of Howard’s policy improvement. Then the raw networks are refined on grid via a few more iterations to sharpen the non-smooth regions and kinks in value and policy. See section 3.4 for the detailed refinement step.

We must monitor *out-of-sample* convergence metrics to evaluate the final *refined* networks:

- Bellman residual squared from (eqn 3.12)
- Bellman right-hand-side from (eqn 3.14)
- Bond pricing residual squared from (eqn 3.8)
- Mean absolute errors between the default probability formula (eqn 3.7) and the true probability computed from Tauchen transition matrix on discrete grids (eqn 3.2)

We say that the algorithm converges if and only if the *out-of-sample* Bellman residual, bond pricing residual, and the default probability error are below tolerance ϵ_{tol} ; and that there is no other sampled actions that increase the Bellman right-hand-side by ϵ_{tol} .

¹Here an *epoch* is a scheduling unit, a block of 500 gradient steps sharing one target network and one learning rate, not a pass over a fixed dataset: every step draws a fresh i.i.d. mini-batch.

Proposition 3.2: Correctness of neural policy iteration

Over the continuous state $\times$ parameter space, the refined networks $(V_\phi, h_\phi^i, h_\phi^b, h_\phi^c)$ and the implied default probability $P(z, k', b' - c')$ and equilibrium bond price $q(z, k', b' - c')$ converge to an approximated local optimum solutions of the risky debt model (Proposition 2.1), if and only if the four convergence metrics pass the rule with tolerance ϵ_{mpi} .

3.2.5 Implementation notes

Gradient steps. Each gradient step is the literal policy-iteration pair: a value update, then a policy update, on separate Adam optimizers and separate gradient tapes. The value step freezes the policy weights; the policy step freezes the value weights and reuses the just-updated value network.

```
with tf.GradientTape() as tape:                                # Step 2: value update
    residual, _, _ = residual_and_rhs(bundle, batch, ...)
    loss_v = tf.reduce_mean(tf.square(residual))
    value_opt.apply_gradients(zip(tape.gradient(loss_v, value_vars), value_vars))

with tf.GradientTape() as tape:                                # Step 3: policy update
    _, rhs, _ = residual_and_rhs(bundle, batch, ...)           # uses the updated value net
    loss_pi = -tf.reduce_mean(rhs)
    policy_opt.apply_gradients(zip(tape.gradient(loss_pi, policy_vars), policy_vars))
```

Forward-mode auto-differentiation for the Taylor coefficients. To compute default probability, we need the slope $a = dV'/d\varepsilon$ and curvature $c = d^2V'/d\varepsilon^2$ of the target value along the shock at $\varepsilon = 0$. Forward-mode auto-differentiation computes these by carrying, alongside every quantity x in the computation, its derivative $\dot{x} = dx/d\varepsilon$, and propagating the pair $(x, \dot{x})$ through each operation by the chain rule: a constant maps to $(a, 0)$, a sum to $(x_1 + x_2, \dot{x}_1 + \dot{x}_2)$, and a function to $(f(x), f'(x)\dot{x})$. Because $\dot{\varepsilon} = d\varepsilon/d\varepsilon = 1$ and every rule is the chain rule, the carried companion at the output gives exactly the derivative we want.

TensorFlow provides this as the built-in `tf.autodiff.ForwardAccumulator`: one tells it which variable to differentiate with respect to, and its method `jvp(y)` returns the carried derivative of y . The smallest example makes the operation concrete:

```
x = tf.constant(3.0)
with tf.autodiff.ForwardAccumulator(primals=x, tangents=1.0) as acc:
    y = x * x           # ordinary value: y = 9.0
    acc.jvp(y)           # carried derivative dy/dx = 2x = 6.0
```

The accumulator carried $\dot{x} = 1$ forward through $y = x*x$, applied the product rule to get $\dot{y} = 2x\dot{x} = 6$, and `acc.jvp(y)` read it back. In our case the differentiation variable is the shock ε , so `acc1.jvp(vprime)` returns $dV'/d\varepsilon = a$; the curvature is the derivative of a , obtained by wrapping the computation of a in a second accumulator and reading off `acc2.jvp(a) = d^2V'/d\varepsilon^2 = c`:

```

eps      = tf.zeros_like(logz)    # evaluate at eps = 0, one shock per sampled state
tangent  = tf.ones_like(eps)
with tf.autodiff.ForwardAccumulator(eps, tangent) as acc2:
    with tf.autodiff.ForwardAccumulator(eps, tangent) as acc1:
        logz_p = rho * logz + sigma * eps
        vprime = target_net(normalize_next(logz_p, kprime, bpp), param_norm)
        a = acc1.jvp(vprime)      # dV'/deps
    c = acc2.jvp(a)               # d2V'/deps2

```

The shock ε is a length- N vector (one per sampled state) carried through in a single pass, which is valid because each sample’s value depends only on its own shock. The whole block runs underneath the reverse-mode policy-gradient tape of Step 3; building a, c in forward mode keeps that outer pass robust. Otherwise if we add a second pair of nested reverse tapes, the nested reverse-mode accumulation can hit an empty-gradient shape error on batches whose default region is empty.

Target value network. The value-price circularity is broken by using a target value network, defined as lagged copy of the current value network. In current implementation, the target network weights are frozen throughout an epoch (500 gradient steps, each step draw i.i.d. fresh sample), and updated only at the beginning of each epoch. This design choice is to prioritize stability during training, but convergence may require large number of steps. Alternative schedule can update the target value more frequently (e.g., 100 gradient steps) or use polyak averaging $V^- \leftarrow (1 - \rho)V^- + \rho V$. We should always monitor the pricing equation residual as diagnostic metric.

Keeping the equilibrium price channel alive. The firm prices its own default risk only if, during policy improvement, the gradient can travel the full chain from the objective back to the actions: the payout D depends on the bond price q ; q depends on the default probability P_{def} (and, directly, on the actions through the recovery value and the gate); P_{def} is the closed-form function of the value level, slope, and curvature (V_0, a, c) of equation 3.7; and those three come from differentiating the *target* value network V^- evaluated at the next-period state $(k', b' - c')$. The target network’s *weights* are held fixed for the epoch, but its *inputs* $(k', b' - c')$ are functions of the current actions, so the gradient rides the inputs, not the weights. If any link in this chain is cut, the firm stops seeing how its leverage moves the price and over-issues debt, while the Bellman residual can still look healthy. Four failures break the chain quietly; the first three are quick to state:

- **Detaching the price.** Stopping the gradient on q (a natural-looking mistake, since the target weights are frozen anyway) cuts the action-to-price link. Training still runs and the residual still falls, but the firm now treats the price as a constant and over-levers.
- **A dormant start.** From the low-debt initialization the chosen debt is fully covered by recovery, so the gate $g = \mathbf{1}\{R < b'k'\}$ is zero, q is the risk-free price, and the channel is correctly switched off. The firm must first lever into risky debt for the channel to turn on. Too few epochs may not be sufficient for training to visit default states.
- **Eager execution.** The curvature c relies on the forward-mode differentiation of the previous note, and differentiating that for the policy gradient only works when the training step is compiled into a graph (`tf.function`); run line-by-line (eager) it raises an error.

We guard each of these. The price keeps its dependence on the actions (no stop-gradient on q); only the target *weights* are frozen. The default-probability formula clamps the discriminant and chooses

cases so its gradient stays finite even where there is no default. The training step is always compiled. We log the gate-active fraction and leverage each epoch, so a dormant channel is visible after training rather than hidden. And a regression test asserts the channel is actually live: in a risky-debt regime, removing the price’s dependence on the actions must change the debt-policy gradient.

Non-finite gradients. The default-probability cases in equation 3.7 are hard-coded with `tf.where`, not a Python `if`. `tf.where` always evaluates and differentiates *every* branch, so each branch must be computed from finite inputs even when it will be discarded. The forward pass computes the discriminant Δ (`disc`), then $\sqrt{\Delta}$, then the roots $\varepsilon_{\pm}$, and finally the `P_def` values; the backward pass walks those steps in reverse, from `P_def` back to `disc`, to produce the gradient. On a row with $\Delta < 0$ a naive $\sqrt{\Delta}$ is `NaN`, so the discarded root branch carries `NaN` values. The select still returns the correct value, `P_def = where(has_roots=False, NaN, 0) = 0`, but in the backward pass the zero gradient routed into that branch is multiplied by its `NaN` derivatives, and `0 * NaN = NaN` poisons the gradient for the whole mini-batch. The fix is to keep every branch finite: the code takes $\sqrt{\max(\Delta, 0)}$ (so $\Delta < 0$ gives $\sqrt{0} = 0$, not `NaN`) and divides by a guarded denominator in the linear ($c \approx 0$) case, leaving the value unchanged and the gradient finite.

Scale of the value target. The value network outputs equity value directly, with no rescaling, and equity value spans orders of magnitude across the box: a large k -firm is worth far more than a small one, and the typical scale also shifts across parameter vectors (e.g., high variance σ). The training objective is the mean squared Bellman residual (equation 3.12), so it is dominated by the high-value state-parameter vectors, while low-value state-parameter vectors contribute little and can stay poorly fit even when the average loss is small. This matters because the solver is meant to be accurate everywhere in the box for estimation, so accuracy that is good only where values are large is a hidden bias. We monitor it with the *relative* Bellman residual: $|V - RHS|/(|V| + \epsilon)$. This diagnostic is computed by parameter vector and by value decile, so that we can monitor whether training quality varies across different regions in the state-parameter box.

Extrapolation in the expected equity value. The expectation step evaluates the value network at next-period productivity placed at the quadrature nodes (equation 3.5), and those nodes can fall outside the ± 2.5 standard-deviation box on which the network was trained and normalized. This matters because a neural network is unreliable beyond the range it saw in training, and these out-of-range values feed directly into the Bellman target, so their error propagates back into the learned value. We monitor it by logging the fraction of continuation nodes whose normalized productivity exceeds one in magnitude, together with the largest such magnitude. A direct fix in future is to widen the productivity sampling and normalization range to about ± 4 standard deviations, which is enough to contain every node (the farthest needs about 3.8) and covers essentially all of the shock mass.

Smoothing temperature tuning. The hard indicators and kinks in the payout are replaced by smooth approximations controlled by a temperature parameter κ (equation 3.9), and the choice of κ is a tradeoff between stable gradient and gate sharpness. A smaller κ makes the smooth gate follow the true hard gate closely, so the payoff is accurate, but the gate turns very steep near its switch and produces large, spiky gradients there that can destabilize the optimizer. A larger κ spreads the gate over a wider band, so the gradients are gentle and training is stable, but the payoff is biased away from the true kink. There is no universally correct value. It is left to the user’s discretion to balance payoff accuracy against gradient smoothness.

Ergodic coverage of the sampling box. Training states are drawn uniformly over the whole state-by-parameter box (Step 1). This is what makes the trained networks universal, but it also spends effort on combinations the model never reaches in the high-dimensional box. Regions such as high (k', b') and low z' is far from economic steady states, yet they are sampled as frequent as the ergodic regions the firm visits a lot. This matters because capacity and gradient steps spent on never-visited regions are wasted, and, worse, the part of the box where the simulated moments for estimation actually live (the ergodic set) can end up underfit even when the average loss over the box is small. We monitor it by simulating the model at representative parameter vectors to obtain the ergodic distribution of states, then reporting (i) the fraction of uniform draws that land in that ergodic region, where a low fraction signals wasted effort, and (ii) the Bellman residual on ergodic samples compared with the residual on the uniform box. If the ergodic residual is the smaller of the two the waste is harmless; if it is larger, the region that matters for estimation is underfit and the sampling should be revisited (for example by mixing in simulated states). We report this diagnostic before deciding on any change.

3.3 Third solver (to be specified)

3.3.1 Purpose and where it fits

[To be written.]

3.3.2 Theory and derivation

[To be written.]

3.3.3 Algorithm

[To be written.]

3.3.4 Implementation (data structures, TensorFlow specifics)

[To be written.]

3.3.5 Configuration and hyperparameters

[To be written.]

3.3.6 Accuracy and validation against the benchmark

[To be written.]

3.3.7 Limitations and resolved design decisions

[To be written.]

3.4 Network refinement on discrete grids

This section describes how the raw policy and value networks from the neural policy iteration or the Euler residual method are refined on a discrete grid, validated against the Howard’s policy iteration (HPI) benchmark, and then certified by a set of out-of-sample convergence metrics, before being passed to the simulation and estimation block.

3.4.1 Why a refinement step

The trained networks are *smooth* approximators, and they are trained against deliberately smoothed objects: the net payout (reward) are approximated with smooth surrogate, the conditional expectation is a Gauss-Hermite quadrature, and the default probability is computed analytically based on a quadratic Taylor expansion. These choices are what make the training differentiable, but they also compromise the non-smooth features of the true model solution, such as the default boundary at $V = 0$, the jump discontinuities in issuance and adjustment costs, and the inaction regions in the optimal policy.

This on-grid refinement step has two key roles: (1) it restore the genuinely non-smooth parts of the optimal policy and value function; and (2) it provides an additional “safeguard” layer where convergence to the true model solution has mathematical theorem guarantee. This improves credibility and our confidence in the neural network approximations (Proposition 3.1).

The refinement step is simple. It projects the network solution onto a discrete grid and runs a few more rounds of *exact* Howard policy improvement (described in Section 3.1). Because the network already provides a good warm start, we can afford using *dense* grids to improve precision by combining:

- *local* policy search with *fewer* action grids to be evaluated
- *fewer* rounds of value iteration till convergence

The value and policy networks are universal over the entire state-parameter space $(z, k, b - c) \times \Theta$, so that the relatively cheap refinement step can be re-run quickly for each new Θ evaluation. The refined grid solution conditional on a specific Θ is what the simulation and estimation block consumes.

The same refinement procedure is a drop-in module after either the neural policy iteration or the Euler residual method, as both methods output neural networks to approximate value and policy (and price). Note that refinement always recomputes the default probability and bond price exactly.

3.4.2 Refinement algorithm

Refinement runs per parameter vector Θ , on the same grid as the HPI benchmark: productivity z on the n_z Tauchen nodes, capital k on a log-spaced grid, net debt b on a uniform grid, and the controls (k', b', c') on their own finer grids. All arrays are double precision on the CPU.

Step 0: Evaluate the networks on the grid. Query the trained value and policy networks at every grid state (z_j, k, b) to obtain the value surface V_{net} and the policy actions (i, b', c') .

Step 1: Initialize. Set the value iterate to the equity value $V^{(0)} = \max\{V_{\text{net}}, 0\}$. Convert each network action to its nearest control-grid index: the next-capital index from $k' = (1 + i - \delta)k$, the gross-debt index from b' , and the cash index from c' .

Step 2: Refine for R rounds. Repeat $R = 6$ times:

1. *Local policy improvement.* At each grid state, build a small candidate set for each control: its current index, the indices chosen at the six immediate (z, k, b) neighbors, and the two ± 1 index perturbations, which gives nine candidates per control and hence $9^3 = 729$ joint candidates. Evaluate the exact Bellman right-hand side at all candidates and keep the maximizer. The network warm start is what makes this local search sufficient: the optimum is near the network policy, so a global grid search is unnecessary.
2. *Policy evaluation.* Holding the policy fixed, solve the value fixed point exactly. The default method linearizes the Bellman operator on the current *solvent set* (the next-period states with $V' > 0$) and solves the dense linear system $(I - \beta A)V = D$, where $\beta = 1/(1 + r_f)$, A is the interpolation-weighted transition restricted to the solvent set, and D is the exact dividend. The bond price, and hence D , is held fixed during this solve and recomputed between rounds, which keeps the system linear. The solvent set is then updated and the solve repeated until the set stops changing. (A value-iteration variant that recomputes the price every sweep is kept as a cross-check.)

Step 3: Output. Return the refined value (the equity value $\max\{V, 0\}$ reported to shareholders, and the raw V whose negative region marks the default states), together with the refined policies (i, k', b', c') on the grid.

The Bellman right-hand side used in both sub-steps is the exact one: the continuation is the Tauchen-weighted expected equity value (equation 3.1), the default probability is the Tauchen-weighted share of defaulting next-period states (equation 3.2), the bond price follows from the closed form (equation 3.8), and the dividend is the exact, non-smoothed payout. Off-grid next states $(k', b' - c')$ are evaluated by bilinear interpolation in $(\log k, b)$, while next-period productivity z' stays on the Tauchen nodes.

3.4.3 Validation against the benchmark

The refined solution is compared on the grid against the HPI benchmark, which carries the fixed-point guarantee. We report the agreement of the value and policy surfaces (correlation and maximum absolute difference) and the following hard mechanics checks on the refined solution, each evaluated with the exact objects:

- the *Bellman residual* $V - \text{RHS}$ at the chosen action, on the solvent set, is below tolerance relative to the value scale;
- the bond price q lies in $[0, 1/(1 + r_f(1 - \tau))]$, that is between zero and the risk-free price;
- the accounting identity $k' = (1 + i - \delta)k$ holds for the reported investment rate.

Passing these on the grid is the concrete content of the claim, in the correctness discussion of the neural policy iteration, that the trained solution is first validated against Howard’s policy iteration.

3.4.4 Out-of-sample convergence metrics

The grid checks above certify the solution *at the grid points*. To certify it as a function over the continuous state and parameter space, we evaluate a set of *out-of-sample* metrics on a fresh batch of states and parameter vectors, drawn the same way as in training (uniform over the box) but with a different seed, so the numbers are not optimistic in-sample values. All metrics use the *exact* objects (exact dividend, hard indicators, and a fine expectation), not the smoothed training surrogates, so

they measure violations of the true model conditions. Write $s = (z, k, b; \Theta)$ for a sampled point, V for the value being evaluated, and $a = (i, b', c')$ for its policy action with $k' = (1 + i - \delta)k$.

Bellman residual. The value must satisfy the Bellman equation at its own action,

$$r(s) = V(s) - \left[D(s, a, q) + \frac{1}{1+r_f} \mathbb{E}_{z'|z} \max\{V(z', k', b' - c'; \Theta), 0\} \right], \quad (3.15)$$

and we report the scale-relative root-mean-square and maximum, $(\text{mean}_s r(s)^2 / (V(s)^2 + \epsilon_0))^{1/2}$ and $\max_s |r(s)| / (|V(s)| + \epsilon_0)$. The scale-relative form is used because equity value spans orders of magnitude across the box (see the value-scale note in the method section), so an absolute residual would be dominated by the high-value states.

Default-probability residual. The analytic default probability of equation 3.7 is compared, at the action's $(k', b' - c')$, against a fine reference probability $P_{\text{def}}^{\text{ref}}$ computed by the Tauchen-weighted count of defaulting next-period states on a dense productivity grid (equation 3.2),

$$e_P(s) = |P_{\text{def}}^{\text{formula}}(s) - P_{\text{def}}^{\text{ref}}(s)|, \quad (3.16)$$

reported as a mean and a maximum. This isolates the error of the quadratic Taylor approximation; it is expected to be small in the interior and largest near the default boundary.

Bond-pricing residual. The price the firm acted on must be consistent with its true default risk. Plugging the reference default probability into the pricing formula (equation 3.8) gives the consistent price $q^{\text{ref}}(s)$, and the pricing residual is

$$e_q(s) = |q(s) - q^{\text{ref}}(s)|, \quad (3.17)$$

again reported as a mean and a maximum. This residual is driven by the default-probability residual above and by the value error that feeds it.

Policy-maximization gap. The policy must locally maximize the Bellman right-hand side. At each sampled state we draw a set $\mathcal{A}(s)$ of alternative feasible actions (perturbations of a within the feasible box, respecting the k -dependent investment bounds and $b' \in [0, \bar{b}']$, $c' \in [0, \bar{c}']$), evaluate the exact right-hand side at each, and record the suboptimality gap

$$\Delta(s) = \max_{a' \in \mathcal{A}(s)} \text{RHS}(s, a') - \text{RHS}(s, a), \quad (3.18)$$

reported as a mean positive gap and a maximum. A non-positive gap means no sampled alternative beats the policy at that state.

Convergence criterion. Consistent with the correctness criterion of the neural policy iteration, we declare the solution converged when the out-of-sample Bellman residual (equation 3.15), the bond-pricing residual (equation 3.17), and the default-probability residual (equation 3.16) are all below a tolerance ϵ_{tol} , and the policy-maximization gap (equation 3.18) does not exceed ϵ_{tol} , that is no sampled action raises the right-hand side by more than ϵ_{tol} . Because there is no contraction guarantee, this is an empirical certificate measured on held-out points, not a theorem.

Chapter 4

Model Estimation

[To be written: the ESTIMATION CONTRACT. Each method consumes a solved model (CH3) and emits parameter estimates with uncertainty. Simulation is method-specific and lives inside each method; the only shared primitive is the mechanical "roll a path forward given (policy, shocks, parameters)", reused by every method.]

4.1 Simulated Method of Moments (SMM)

4.1.1 Purpose and where it fits

[To be written.]

4.1.2 Simulation strategy

[To be written: how this method generates / uses simulated data, including shock sampling and the RNG / common-random-numbers scheme.]

4.1.3 Objective and inference

[To be written.]

4.1.4 Algorithm

[To be written.]

4.1.5 Implementation

[To be written.]

4.1.6 Configuration and hyperparameters

[To be written.]

4.1.7 Identification and diagnostics

[To be written.]

4.1.8 Validation and recovery

[To be written.]

4.1.9 Limitations and resolved design decisions

[To be written.]

4.2 Bayesian estimation

4.2.1 Purpose and where it fits

[To be written.]

4.2.2 Simulation strategy

[To be written: how this method generates / uses simulated data, including shock sampling and the RNG / common-random-numbers scheme.]

4.2.3 Objective and inference

[To be written.]

4.2.4 Algorithm

[To be written.]

4.2.5 Implementation

[To be written.]

4.2.6 Configuration and hyperparameters

[To be written.]

4.2.7 Identification and diagnostics

[To be written.]

4.2.8 Validation and recovery

[To be written.]

4.2.9 Limitations and resolved design decisions

[To be written.]

Chapter 5

Validation and Diagnostics

The pipeline needs tests that check whether its outputs are correct, not only that it runs. Three categories cover this:

- Oracle (known-answer): run on a case with a known answer (a trusted alternative method such as VFI, or data simulated from known true parameters) and check the pipeline recovers it. Sections 5.2 and 5.3.
- Economic property: check properties that theory says must hold for any input, including parameter comparative statics. Section 5.4.
- Regression: freeze a validated run under a fixed seed and pinned versions and check that later runs still match. This catches drift, not correctness. Section 5.5.

Section 5.1 builds the shared VFI benchmark the oracle tests rely on.

5.1 Benchmark solver

Both oracle tests need an independent, trusted solution of the model at a fixed parameter vector. Use value function iteration (VFI) on the discrete grid, built only for validation and never used in the estimation loop.

- Input: one fixed parameter vector β .
- Grids and operators: the state grid ($11 \times 15 \times 35$ over $(\log z, k, b)$) and control grid ($81 \times 91 \times 71$ over (k', b', c')) of Section 3.4, the 11-state Tauchen expectation, the exact (non-smoothed) dividend (Eqs. 7 to 9), and the joint value/bond-price fixed point (recompute q from the current V each sweep, as in Section 3.4 step 3).
- Solve: iterate the Bellman operator from a cold start (set V to the no-default, no-adjustment value, or to 0). At each state node, maximize the right-hand side by a global search over the full control grid. This differs from the grid refinement in Section 3.4, which does a local search around the network policy. Stop when the sup-norm change in V is below 10^{-8} and the argmax policy is unchanged between sweeps. Reuse the Section 3.4 policy-evaluation linear solve for the inner step; only the policy-improvement search is global.
- Output: VFI value and policy arrays on the grid. Off-grid values use the same log- k bilinear interpolation as Section ??.

VFI is intentionally exact and slow. It is the ground truth the network outputs are checked against.

5.2 Oracle test 1: Monte Carlo recovery of parameters and moments

This is the critical end-to-end test. It reproduces DF26 Section 4.1 and its Figures 1 and 2. The idea: build data from known true parameters with the trusted VFI solver, then check that the full pipeline recovers both the moments and the parameters.

Each draw is an independent recovery trial: run the full pipeline (Blocks 1 to 3 and the controller, including adaptive shrinkage toward that draw’s target) end to end per draw. The networks, the collection, and the surrogates depend only on the bounds, not on the target moments, so an efficient implementation may instead train them once over the fixed Table A1 bounds and repeat only the target-dependent Levenberg-Marquardt estimation per draw; this skips per-target shrinkage, so use it only if Figure V1 confirms the full-box surrogate is accurate (R^2 near 1).

Procedure:

1. Draw a true parameter vector β^* uniformly over the Table A1 bounds (Section ??). Fix the seed.
2. Solve the model at β^* with the VFI benchmark (Section 5.1).
3. Simulate a firm panel from the VFI policy using the Section ?? settings (5,000 firms, 300 periods, 200 burn-in, Tauchen chain).
4. Filter: if the panel has no defaulting firms, discard the draw and redraw. The recovery rate χ is not identified without defaults, so DF26 excludes these cases. Keep drawing until you have N kept draws. DF26 used $N = 40$; use 40 as the baseline and more for tighter coverage.
5. Compute the 11 targeted moments (Section ??) from the panel. These are the true targets $m^*(\beta^*)$.
6. Run the Levenberg-Marquardt estimation (Sections ?? to ??) against $m^*(\beta^*)$, giving the estimate $\hat{\beta}$.
7. Refine and simulate at $\hat{\beta}$ (Block 2, Sections 3.4 to ??) to get the fitted moments $\hat{m} = m(\hat{\beta})$.

Repeat steps 1 to 7 for all N kept draws, then produce two figures.

Figure V1 (true versus fitted moments, reproduces DF26 Figure 1). Eleven panels, one per moment in the Section ?? order. In each panel the x-axis is the true target moment m_j^* across draws and the y-axis is the fitted moment $\hat{m}_j$; draw each draw as a marker with a same-colour capped vertical spike for its 95% confidence interval (1.96 times the moment’s firm-clustered standard error, Section ??), over a dashed 45-degree line. Fix both axes to a common range with a square box, so the 45-degree line is exactly diagonal and coverage is read off the full range, not an auto-zoom. Title each panel with the moment name and annotate its R^2 , the standard linear-regression coefficient of determination (the squared correlation between true and fitted, in $[0, 1]$). Pass if $R_j^2 \geq 0.99$ for all 11 moments (DF26 reports 1.00).

Figure V2 (true versus fitted parameters, reproduces DF26 Figure 2). Eight panels, one per parameter $(\theta, \rho, \sigma, \delta, \gamma_1, \gamma_0, \chi, c_f)$. In each panel the x-axis is the true β_j^* and the y-axis is the estimate $\hat{\beta}_j$; draw each estimate as a marker with a same-colour capped vertical spike for its 95% confidence interval (1.96 times the across-fold standard deviation of the estimate), over a dashed 45-degree line. Fix both axes to the parameter’s Table A1 range with a square box, so the 45-degree line is exactly diagonal and coverage and bias are visible across the full range. Annotate each panel’s R^2 (the squared correlation between true and fitted, in $[0, 1]$, same definition as Figure V1), and also report the per-parameter bias (mean of $\hat{\beta}_j - \beta_j^*$) and RMSE, which capture the offset and scale that R^2 alone does not. Pass if $R_j^2 \geq 0.95$ for at least seven of the eight parameters (DF26 reports ≥ 0.97 for seven of eight). χ is the expected weak one; a low R^2 there signals weak identification, not a bug.

Repeat the whole exercise under several seeds and report the spread of R^2 across seeds, since the draws and the simulation noise vary. A correctly specified data-generating process should give near perfect recovery; a clear miss flags a pipeline bug rather than a modeling limit.

5.3 Oracle test 2: policy network against benchmark

This checks that the network solution matches the trusted on-grid solution at a fixed parameter vector. It reproduces and extends DF26 Appendix Figure A1.

Fix β at the reference estimates (Section ??), or any interior vector; state which. Build three solutions at that β : the raw Block 1 network policy, the refined network policy (Block 2 grid refinement at β , Section 3.4), and the VFI benchmark policy (Section 5.1).

Reference holding point for one-dimensional slices: median productivity $z = 1$, near-zero net debt $b = -0.03$, a fixed low capital k_{ref} (state the value, for example $k_{\text{ref}} = 0.2$), and the held parameters at the reference β .

Figure V3 (policy slices). A grid of one-dimensional slices. Rows are the policies i , b' , and c' (add V if useful). Columns are the 11 arguments: the three states k , b , z and the eight parameters $\theta, \rho, \sigma, \delta, \gamma_1, \gamma_0, \chi, c_f$. For each (policy, argument) panel, vary that one argument over its range (state ranges from Section ??; parameter ranges from Table A1) on about 50 points, hold every other state and parameter at the reference point, and plot the policy.

- State-argument panels (columns k , b , z): produce one figure per state axis, each a 1x4 panel over V, i, b', c' , so the (policy, state) combinations are complete. Overlay three curves: raw VFI as a thick dashed grey line, the Block 1 network in blue, and the refined network in bold red. Keep the refined curve the most visible, since it is the policy actually used.
- Parameter-argument panels (the eight parameters): produce one 1x3 figure per parameter (columns i , b' , c'), saved separately per parameter. Overlay two curves only, the Block 1 network (blue) and the refined network (red); raw VFI is omitted, because a single VFI solve fixes β and so cannot trace a parameter axis (see the optional extension).

The k column for rows i , b' , c' at $z = 1$ and $b = -0.03$ is exactly DF26 Appendix Figure A1. Reproduce it as the headline panel set, with the same three-curve legend.

Quantitative pass criteria (beyond the visual overlay). Over the state grid in the good region ($V > 0$), compare the refined network policy to VFI: report the maximum and mean absolute deviation per policy. The refinement targets VFI-level accuracy (Section 3.4, stop at 10^{-10}), so the refined policy should sit on the VFI curve; pass if the maximum relative deviation is below 1 percent per policy (or an absolute threshold scaled to each policy's range, stated in the output). Report the raw Block 1 deviation as well for context, with a looser sanity bound (for example within about 10 percent); the raw network is the unrefined approximation and is not required to match VFI tightly.

Optional comprehensive extension (skip at the current stage). To benchmark the parameter-axis panels against VFI, re-solve VFI at each point along a parameter axis (one VFI solve per grid point of that parameter, with the others held), producing a VFI curve for the parameter slices. This is expensive, one full VFI solve per point, but it validates the network's parameter dependence against the trusted method. Add it once the state-axis checks pass.

5.4 Economic property tests

These check properties that must hold for any input, not just on a known-answer case. Groups 1 to 5 are always-true properties; Group 6 is parameter comparative statics. Each check is hard or a diagnostic: a hard check gates the suite (a violation fails the run); a diagnostic is computed and reported but never fails it. Hard checks are reserved for deterministic or theory-certain properties; anything expected but not guaranteed is a diagnostic. Shape and slice checks reuse the one-dimensional slice machinery and reference point of Section 5.3, evaluated on the VFI and refined-network solutions unless noted.

Group 1, correct bounds (all hard; deterministic domain, checked on every evaluation).

- Bond price $q \in [0, 1/(1 + r_f(1 - \tau))]$; the bracket is a repayment fraction in [recovery, 1].
- Default probability $P_{\text{def}} \in [0, 1]$; recovery fraction $R/(b'k') \in [0, 1)$ when the gate $g = 1$.
- Controls land in their boxes: i keeps $k' = (1 + i - \delta)k \in [k_{\min}, k_{\max}]$; $b' \in [0, 2]$; $c' \in [0, 1]$; net debt $b' - c' \in [-1, 2]$. Checking the realized values catches a broken Eq. 26 mapping.
- Tauchen transition matrix: rows non-negative and summing to 1.
- Simulated states stay in the grid box; a leak signals an interpolation or bound bug.

Group 2, correct economics (monotone shape versus states).

Hard assertions (three): V increasing in z ; V decreasing in b ; i decreasing in k . Test: for each, draw about 20 hold configurations (the two held states and β sampled uniformly in the current bounds), sweep the varying state over its grid range on about 50 points, evaluate the VFI and refined solutions, keep interior points with $V > 0$, drop the two endpoints, take consecutive differences, ignore steps with $|\Delta| < \varepsilon_{\text{mono}} = 10^{-3} \times (\text{slice max} - \text{slice min})$, and require every remaining step to carry the asserted sign. Pass means zero violations across all configurations on both VFI and the refined network. The raw Block 1 network is reported, not gated.

Diagnostics (reported, not gated): V versus k (expected up at low k , may flatten or reverse at high k , since net debt is per unit of capital so the debt level scales as $b k$ under decreasing returns); i versus z (expected up); i versus b (expected down).

Skipped (no sign asserted): b' and c' versus every state are ambiguous and hump-shaped (Appendix A1b, A1c); validate their shape through the Section 5.3 overlays instead.

Group 3, correct mechanics (optimality and self-consistency).

Hard checks: the Bellman residual $R = V - D - (1/(1 + r_f)) \mathbb{E}[\max\{V', 0\}]$ (Eq. 27) is small on the refined solution (tight, since refinement converges); the bond-price residual $|q - q(V)|$ (Eq. 6 at the solved V) is small on the refined solution; the accounting identities hold exactly when recomputed and compared ($k' = (1 + i - \delta)k$, next net debt $= b' - c'$, D from Eqs. 7 to 9 equals the D used, R equals Eq. 5); the weighting matrix $\hat{\Sigma}$ is symmetric positive definite (Cholesky succeeds); and no value, policy, price, moment, Jacobian, or weighting entry is NaN or infinite.

Diagnostics: the Bellman and bond residuals on the trained network (looser, since it is approximate); the Levenberg-Marquardt first-order condition (gradient near zero at $\hat{\beta}$, and the objective below nearby perturbations, which reads as flat for weakly identified directions like χ); and the policy-improvement objective (Eq. 29 right-hand side) having plateaued.

Group 4, corner cases (the deterministic checks above, evaluated at named extremes).

Hard checks: in the no-default region (low leverage or high χ , so the gate $g = 0$), q equals the

risk-free price $1/(1 + r_f(1 - \tau))$ exactly, and a simulation with zero defaults runs to finite output without error; under high leverage ($R < b'k'$, $g = 1$), q stays in (0, risk-free) with recovery below 1; at every Table A1 corner the production constants are finite ($\nu = 1 - (1 - \alpha)\theta > 0$, so ξ and A_π finite), $k_{\min} > 0$ so per-capital terms are safe, and the Tauchen grid still covers the high- σ , high- ρ case.

Diagnostics: when the optimal control sits at a bound ($c' \rightarrow 0$, $b' \rightarrow 0$, or i at its limit), the sigmoid policy saturates near the edge while the grid refinement reaches the true edge, so report their agreement; and in the no-default case, report the χ identification flag (Section ??) rather than gating on it.

Group 5, correct consistency (two independent routes agree).

Hard check: surrogate moments versus simulate-then-measure at the same β , where the out-of-sample R^2 (Section ??) is at or above its acceptance threshold; a low R^2 invalidates the estimation.

Diagnostics: the training Taylor P_{def} versus the exact grid P_{def} (report the gap, expected small in the interior and largest near the default boundary, consistent with the paper treating the grid value as the accurate one); the Gauss-Hermite continuation versus the Tauchen continuation at the same state; and, optionally, normalization invariance (the network output at a fixed raw (z, k, b, β) is near-invariant to the active bounds on their overlap), which catches renormalization bugs.

Group 6, correct comparative statics (parameter monotonicities; all hard, no diagnostics).

Assert, holding the other seven parameters and the reference state fixed:

- V decreasing in c_f , in δ , in γ_1 , and in γ_0 .
- V increasing in χ .
- q non-decreasing in χ at fixed state, choices, and P_{def} .

Test: evaluate the Block 1 value network's V at the Section 5.3 reference state across about 20 hold configurations (the held state and the other seven parameters sampled uniformly in the current bounds), sweep the one parameter over its Table A1 range on about 50 points, take consecutive differences, ignore steps with $|\Delta| < \varepsilon_{\text{mono}} = 10^{-3} \times (\text{sweep max} - \text{sweep min})$, and require every remaining step to carry the asserted sign. For q , evaluate Eq. 6 along the χ axis at fixed choices and P_{def} .

These are the only certain parameter signs. The value is a maximum, so it inherits any parameter shift that moves the payoff one way for every policy (a dominance argument needing no closed form), and the tax enters only through the bond-price debt term, so no profit-tax or depreciation shield offsets the cost and depreciation signs; q is a direct function of recovery $R = c'k' + \chi(1 - \delta)k'$. Control and moment signs are not asserted, because the controls are the maximizer, with competing channels and non-convex frictions that leave them ambiguous, and the moments add the stationary distribution; those are covered by the oracle recovery test (Section 5.2) and the Section 5.3 overlays. The value network evaluates any β cheaply, so a violation flags a bug; VFI along a parameter axis can confirm a few points but is not required.

5.5 Regression tests

This is a thin drift guard, not a correctness test; correctness is covered by the oracle and economic property tests above. It does not re-run training, which is slow and, under the async design, not

reproducible (Section ??). Instead it freezes one validated run and re-checks only the deterministic, fast pieces.

Freeze once, from a run that passed the oracle and economic property tests, and commit as small files: the trained network checkpoint (value, three policies, twelve generators) and the surrogate checkpoint; a handful of fixed evaluation inputs (z, k, b, β) ; one parameter vector β_0 with its target moments; and the master seed.

The test reloads these and recomputes, comparing each output to its frozen golden:

- the network forward pass at the fixed inputs (V and the three controls), guarding the network, FiLM, and normalization code;
- the bond price q and dividend D at the fixed inputs via Eqs. 6 to 9, guarding the economic primitives;
- one on-grid refinement sweep from the frozen (V, q, policy) , guarding the Tauchen expectation, the semismooth-Newton step, and GMRES;
- a short fixed-seed simulation from the frozen policy (a few hundred firms, short horizon) and its moments, guarding the simulation and moment code;
- one LM solve on the frozen surrogate against β_0 , guarding the Jacobian, the weighting matrix, and the optimizer.

Each piece is a forward evaluation or a single operator application, so the whole test runs in seconds. Run it on a fixed machine with pinned versions, single-GPU and serial, with op-level determinism on (Section ??), so the float math is repeatable. Compare floats at $\text{rtol } 10^{-5}$, $\text{atol } 10^{-7}$; integer and structural outputs, such as default counts and the active-set partition, must match exactly.

When a change or a version bump alters the goldens on purpose, regenerate them under the same seed and commit them with a one-line note. The test then flags only unintended drift.

Optional: to also guard the training loop, add a short deterministic training slice, a handful of gradient steps from a fixed init and seed in the serial single-GPU mode, and freeze the loss trajectory.

5.6 Value and policy accuracy against VFI

Figure 5.1 reports the Block 1 policy-recovery test (Section 5.3): after grid refinement, the network value and policy functions reproduce the VFI benchmark across the state space.

5.7 Monte Carlo recovery results

Figures 5.2 and 5.3, with Tables 5.1 and 5.2, report the Monte Carlo recovery test (Section 5.2): data are simulated from known parameters, re-estimated through the full pipeline, and compared to the truth.

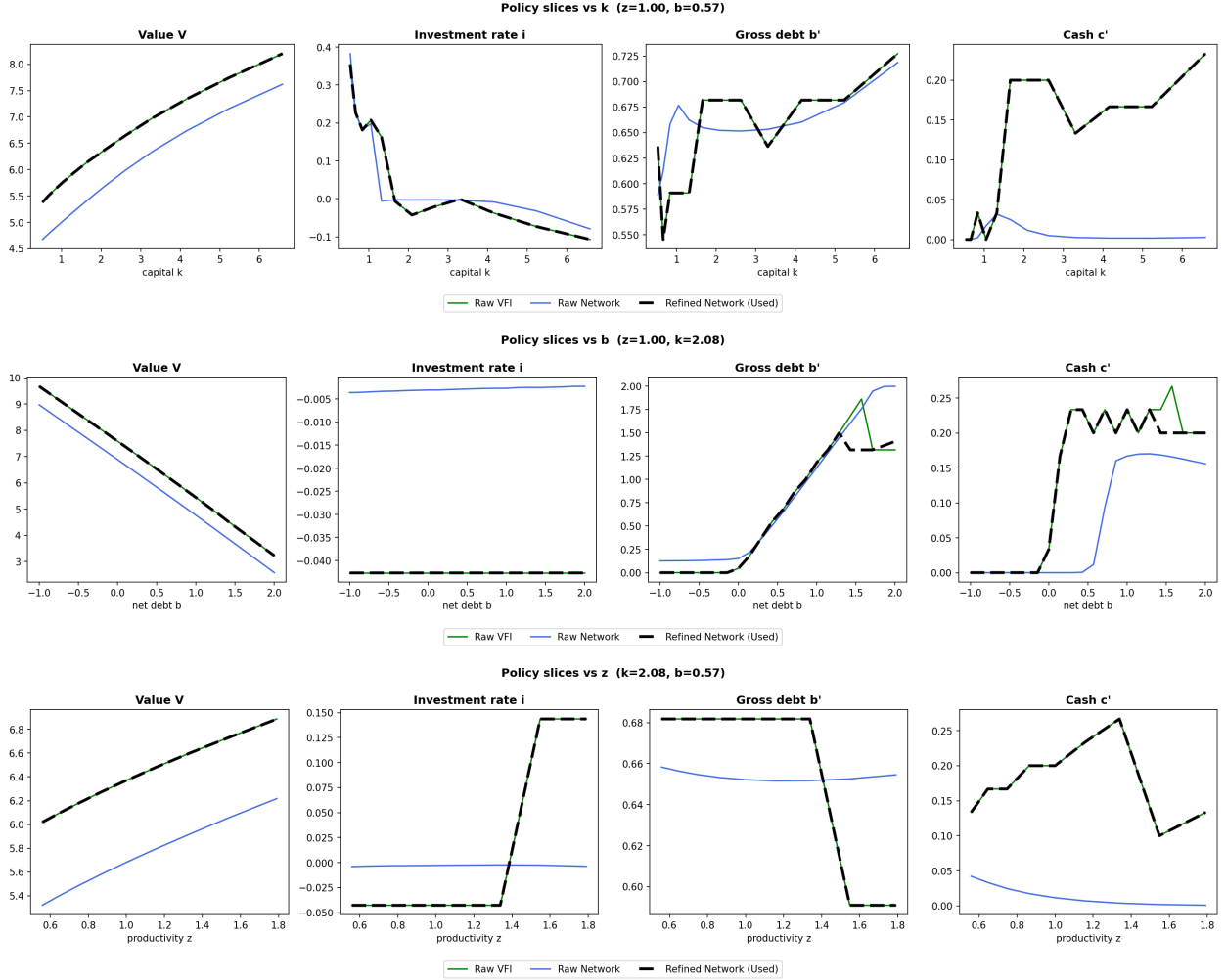


Figure 5.1: Value and policy function slices: refined network versus VFI benchmark.

Notes: Each row varies one state variable (capital k , debt b , productivity z) with the others held at reference values; columns are the value V , investment rate i , gross debt b' , and cash c' . The refined network (used downstream) overlays the grid VFI benchmark; the raw network is shown for comparison.

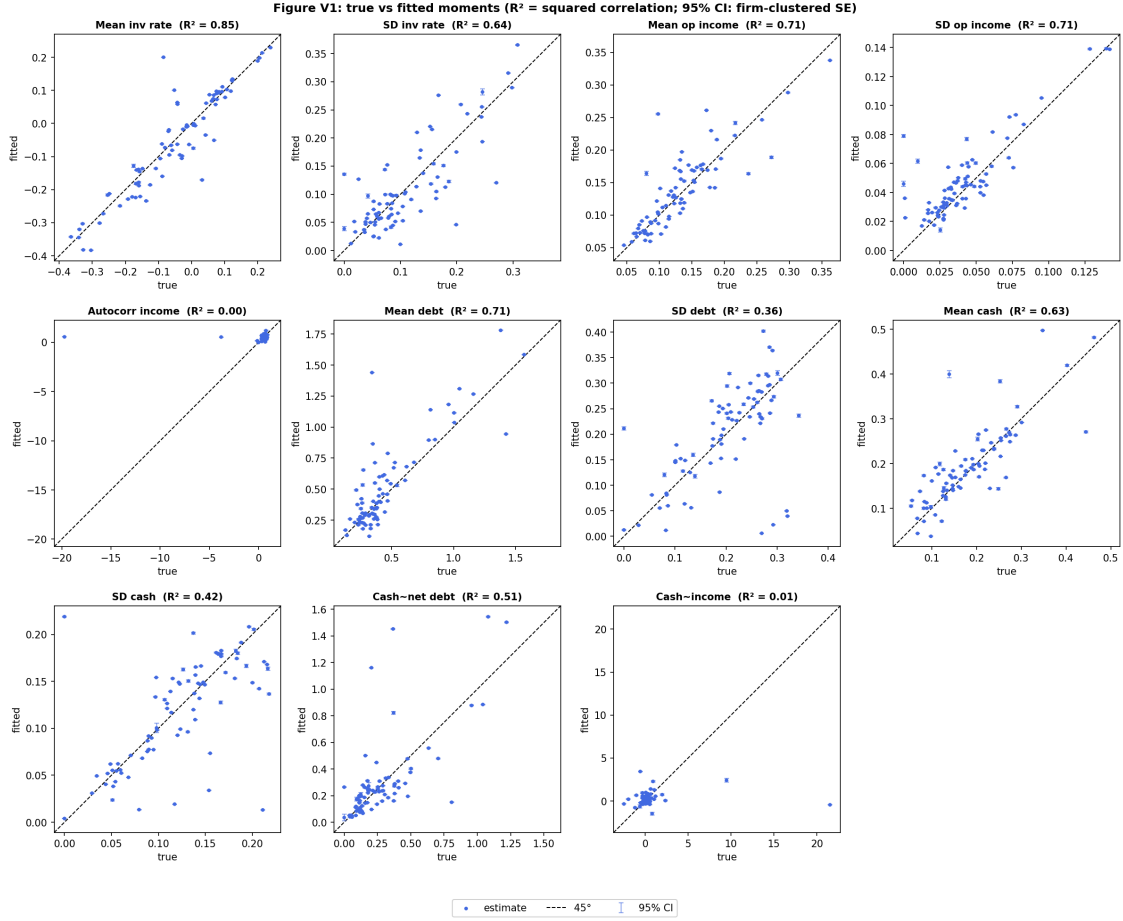


Figure 5.2: Moment recovery: fitted versus true moments across replications.

Notes: Each panel plots the fitted (model-implied) value of one of the 11 target moments against its true value across Monte Carlo replications; the dashed line is perfect recovery (45°) and R^2 in each panel title is the squared correlation. Per-moment values are in Table 5.1.

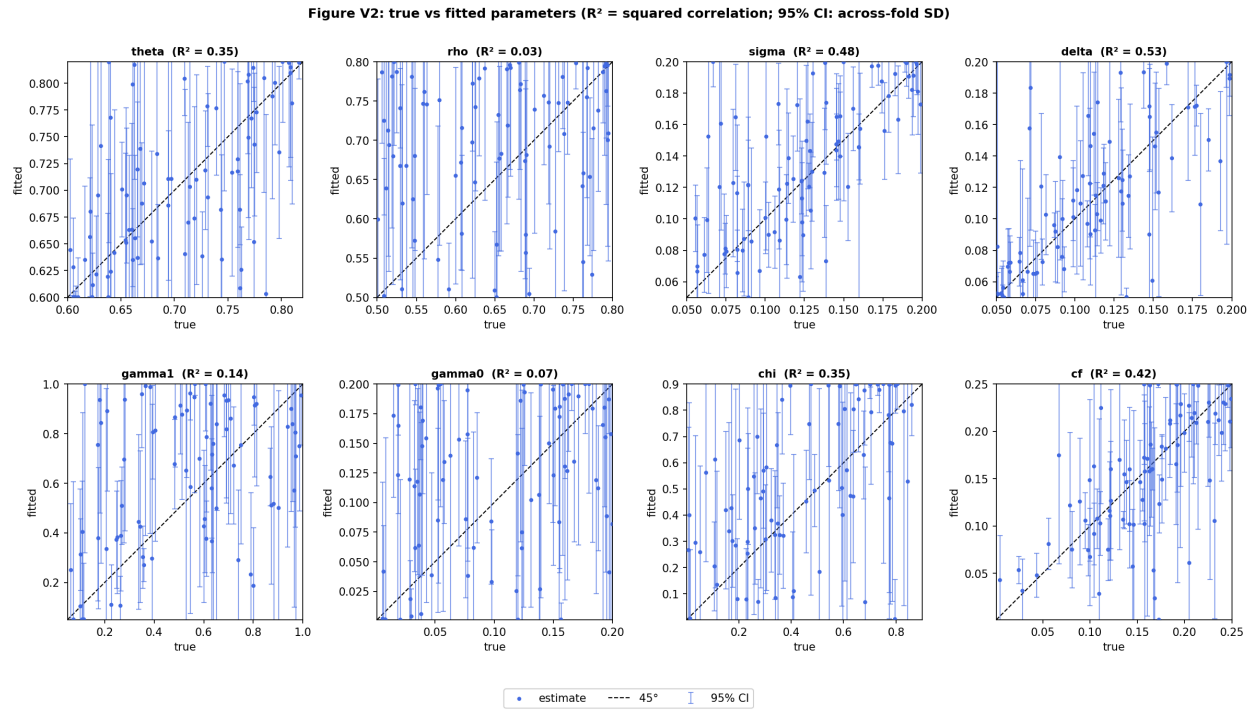


Figure 5.3: Parameter recovery: estimated versus true parameters across replications.

Notes: Each panel plots the estimate of one of the eight structural parameters against its true value, with 95% confidence intervals; the dashed line is perfect recovery and R^2 in each panel title is the squared correlation. Per-parameter values are in Table 5.2.

Table 5.1: Moment recovery summary.

Moment	True	Fitted	95% CI	R^2
Mean inv rate	-0.051	-0.056	[-0.057, -0.056]	0.85
SD inv rate	0.111	0.110	[0.109, 0.111]	0.64
Mean op income	0.131	0.137	[0.136, 0.137]	0.71
SD op income	0.041	0.047	[0.047, 0.048]	0.71
Autocorr income	0.251	0.567	[0.558, 0.577]	0.00
Mean debt	0.454	0.512	[0.510, 0.514]	0.71
SD debt	0.202	0.202	[0.201, 0.203]	0.36
Mean cash	0.183	0.195	[0.194, 0.196]	0.63
SD cash	0.123	0.117	[0.117, 0.118]	0.42
Cash~net debt	0.270	0.293	[0.290, 0.296]	0.51
Cash~income	0.514	0.231	[0.202, 0.260]	0.01

Notes: Mean across Monte Carlo replications of each of the 11 target moments: true value, fitted (model-implied) value, 95% confidence interval (firm-clustered standard errors), and R^2 (squared correlation between fitted and true). Corresponds to Figure 5.2.

Table 5.2: Parameter recovery summary.

Parameter	True	Estimate	RMSE	95% CI	R^2
θ (returns to scale)	0.703	0.703	0.062	[0.610, 0.796]	0.35
ρ (persistence)	0.650	0.693	0.127	[0.547, 0.839]	0.03
σ (shock SD)	0.125	0.135	0.035	[0.097, 0.173]	0.48
δ (depreciation)	0.111	0.113	0.032	[0.076, 0.151]	0.53
γ_1 (convex adj. cost)	0.525	0.636	0.330	[0.278, 0.994]	0.14
γ_0 (fixed adj. cost)	0.104	0.123	0.079	[0.023, 0.223]	0.07
χ (recovery rate)	0.454	0.532	0.252	[0.203, 0.860]	0.35
c_f (fixed op. cost)	0.157	0.153	0.052	[0.087, 0.220]	0.42

Notes: Across Monte Carlo replications for each of the eight structural parameters: true value, mean estimate, root-mean-squared error, 95% confidence interval (across-fold standard deviation), and R^2 (squared correlation between estimate and true). Corresponds to Figure 5.3.

Chapter 6

Commercial Product, Governance, and Practical Considerations

This chapter proposes a variety of commercial products developed from the project. Each section describes one product category: context, potential buyer, commercial value, example usage, and pricing strategy.

Across all product category, the core engine is a *quantitative model and system* consisting of:

- **Information input:** data, parameters, assumptions, theories, configurations;
- **Processing:** machine learning, statistical and econometric methods, numerical optimization;
- **Information output:** quantitative estimates and forecasts.

Importantly, this model does not directly make financial decisions or actions. It is used as a tool to support the user’s judgment and decision-making.

6.1 Product 1: Corporate Finance Advisory (CFA)

Corporate Finance Advisory (CFA) is a global, multi-disciplinary solutions team specializing in structured M&A and capital markets that is focused on differentiating J.P. Morgan’s Investment Banking services. CFA works with firms in all industries and partners across the bank to advise on solutions to problems with multifaceted, complex and unique features.¹

The model typically takes the corporate client’s financial data and other information as input, and produce a credible interval of the optimal capital structure and policies as key outputs. These outputs are further processed to analyze counterfactual scenarios and generate other enhanced insights.

6.1.1 Buyer

Potential buyer is JPMC’s CFA team and similar teams in other banks. This product directly supports:

- **Corporate finance solutions:** Provide analysis and recommendations on capital structure, capital allocation and shareholder distribution policy.

¹<https://www.jpmorgan.com/investment-banking/corporate-finance-advisory>

- **Ratings advisory:** Strategic and tactical advice on management of rating agency relationships and optimization of capital structures for desired credit rating objectives.
- **Structuring and solutions:** Structure products and alternative financing solutions relating to strategic M&A and capital markets.
- **Enhanced insight generation:** Leverage new technologies to improve both data analytics and visualization across all product and coverage groups.

6.1.2 Example

A company holds \$2 billion in surplus cash and wants to return it to shareholders through a buyback, funded partly with new debt. Its board asks one question: does the plan raise value without threatening the company’s credit rating? JPMorgan’s CFA team advises on the answer. The standard approach relies on peer benchmarks, rating-agency ratio scorecards, stress scenarios, and experienced judgment. These can estimate how the company’s ratios map to a likely rating, but they do not value the choice itself, weighing the gain in firm value against the added default risk in a unified, model-based framework.

The model answers the question directly. From the company’s own financials, it simulates the buyback and measures two effects together: the change in firm value and the change in default risk. It shows that the full \$2 billion buyback raises value only modestly and lifts the chance of a one-notch downgrade to roughly 30%. A smaller \$1.5 billion buyback captures most of the value while keeping downgrade risk low. The team brings the board a clear tradeoff, grounded in the company’s own numbers rather than peer averages.

6.1.3 Value and Pricing

CFA work is firm-specific advice on capital structure, allocation, payout, and ratings. The model produces exactly that counterfactual analysis quantitatively. For a banker, this is a differentiated pitch tool: “our proprietary model, estimated on your fundamentals, says X.” This is also a way to standardize work that is currently bespoke and analyst-heavy.

- **Ceiling:** set by how much the bank believes the model lifts mandate wins and fees.
- **Floor:** set by the analyst hours it displaces.

Realistic pricing strategy: platform subscription plus per-user license (seats), with the value carried in the sales narrative rather than a success fee.

Ideal pricing strategy: causal attribution that pins down the success fee per deal (e.g., “our product uplifts 10% of the advisory fee”). We need to deploy causal inference tools for credible evaluation. For example, start with a randomized experiment that gives the tool to different teams internally, collect data on outcomes, and estimate the average treatment effects.

Reason: It is hard to quantify the product uplift of wins and fees. CFA is often a relationship and credibility product that supports the broader IB relationship, so part of the product’s value is indirect and long-term.

6.2 Product 2: Debt Capital Markets (DCM)

J.P. Morgan’s Debt Capital Market team assists clients — including corporations, financial institutions, private equity and strategic investors — on a wide range of debt financing strategies.²

6.2.1 Buyer

Potential buyer is JPMC’s DCM team. This product directly supports:

- **Bond pricing advisory:** provide an independent fair-value anchor based on model estimates from the firm’s fundamentals, which can be used to compare the market spread the bank derived from similar-rated bonds trade.
- **Liability management and structuring:** help the client decide whether to refinance, change the debt mix, or pick a tenor and seniority. Our model enables full dynamic counterfactual analysis across alternative capital structures.

6.2.2 Value

The bank captures this value indirectly. The client receives a better result than a peer-based rule would deliver, which strengthens the relationship and the bank’s case to win the lead role, where several million dollars of underwriting fees are earned. Competing banks, working only from comparables, cannot produce the same fundamentals-based anchor. The tool does not replace the banker’s judgment. It gives that judgment independent support.

6.2.3 Example

A company plans to raise \$1 billion by issuing a bond, and JPMorgan’s DCM team must advise on the price. The standard method anchors the price to where comparable bonds trade, the issuer’s own outstanding debt and similar-rated peers, plus a small new-issue concession. This reflects the market’s current read and the issuer’s rating, not an independent estimate built from the issuer’s own fundamentals.

The model prices the bond from the issuer’s fundamentals. It finds the company is stronger than its rating peers and supports a tighter spread. Carrying that independent anchor into the deal, the bank prices the bond about 15 basis points tighter than comparables alone would have set. On a \$1 billion ten-year bond, that saves the issuer roughly \$1.5 million a year, about \$15 million over its life.

6.2.4 Pricing

DCM decisions attach to basis points on large events (e.g., issuance, M&A). An independent fair-value anchor that helps price a deal a few bps better, or that strengthens the pitch to win the book-runner role, both has a clear and large per-deal value.

Example: A company wants to raise two billion dollars by issuing a 10-year bond. It hires banks to run the sale, and our tool is used by the bank on this deal. There are two places value can come from.

1. **Winning the mandate:** If the tool’s independent fair-value analysis makes the bank’s pitch more convincing, the bank is more likely to be chosen as bookrunner. Suppose the bank’s fee on a deal this size is a few million dollars.

²<https://www.jpmorgan.com/investment-banking/debt-capital-markets>

2. **Better pricing for the client:** Suppose the tool shows the bond can price 5 bps tighter than the comparable-bond view suggested. On two billion dollars, 5 bps is 0.05%, which is one million dollars a year that the borrower saves. This wins more future mandates and builds the bank's reputation.

Pricing strategy: a share of the value the bank can credibly attribute to the tool across a year of deals it touches, where that value is mostly extra mandate fees and proven cost savings.

6.3 Product 3: Leveraged Finance and Private Credit

The model can be used as a fundamentals-based underwriting and monitoring tool for leveraged finance and private credit.

Leveraged finance: J.P. Morgan is the partner of choice for Corporate & Financial Sponsors across the globe with unmatched structuring, marketing and execution capabilities in Leveraged Finance. J.P. Morgan platform is the undisputed market leader in arranging Loans and underwriting bonds that go to fund event-driven financing transactions including Capital Expenditures, Acquisitions, Leveraged Buyouts and Dividend Recapitalizations.

Investment grade finance: J.P. Morgan works directly with issuers, including corporations and financial institutions seeking debt financing. J.P. Morgan advises clients on capital structure strategy, from regular-way bond refinancing or a simple bank loan to multi-billion-dollar capital raises across asset classes. Colleagues partner across capital markets products and country borders to deliver successful structuring, marketing, and pricing

Private credit: J.P. Morgan increases direct lending commitment to 50 billion USD. This strategic move is designed to extend the firm's direct lending capabilities and provide tailored private credit solutions to meet the evolving needs of clients.³

6.3.1 Buyer

Internally it supports JPMorgan's Leveraged Finance group and its direct lending platform in CIB. Externally it is sellable to direct lenders, private credit funds, and leveraged buyout sponsors.

- **Underwriting and leverage sizing:** Give an independent, fundamentals-based estimate of how much debt a borrower can sustainably carry, and its default probability over the life of the loan, as a check on the deal team's comparable-based view before capital is committed.
- **Risk-based pricing:** Produce a fair credit spread for the proposed structure from the borrower's own fundamentals, flagging when a deal underpays the lender for the risk taken.
- **Scenario and covenant stress testing:** Run dynamic counterfactuals across leverage, tenor, and downturn cases, quantifying how default risk and covenant headroom (the leverage and coverage tests) move if earnings fall. This is timely, since 2026 is widely seen as the private credit market's first real stress test.
- **Portfolio monitoring and early warning:** Re-estimate the model each quarter across the existing loan book to flag borrowers whose default risk is rising before it surfaces in the financials, supporting risk governance and investor reporting.

³<https://www.jpmorgan.com/about-us/corporate-news/2025/jpmorgan-increases-direct-lending-commitment-to-50-billion>

6.3.2 Value

The larger value lies in the deals that our tool warns the fund to avoid. A single default on a loan this size can cost the fund \$80 million or more. Avoiding one such loss may outweigh many years of the subscription.

6.3.3 Example

A private credit fund must decide whether to lend \$200 million to a company that a private equity firm is buying. Its analysts compare the deal to how similar companies were financed and conclude the loan is safe and fairly priced. This peer comparison is the industry standard, and it has one structural weakness. No peer company is this borrower carrying this much new debt. The fund's existing tools, comparison tables and rating-style scorecards, all rest on peers and historical averages. None can model this specific borrower under this specific debt load.

Our structural model closes this gap. It uses data of the borrower's own financials and estimates the probability that the company cannot service the debt. It then computes the interest rate that fairly compensates the fund for that risk. In this case, our model confirms the loan is safe but flags the proposed rate as 50 basis points too low. The fund secures the higher rate. On a \$200 million loan, that adds \$1 million of income per year, about \$5 million over a five-year term.

6.4 Model risk and deployment issues

The key issues for product deployment in real life are:

- Model validation, risk management and compliance.
- Both bankers and clients need to trust our model and estimates
- Ongoing monitoring after launch and performance evaluation.

The primary obstacle is **model risk management and compliance**: the product must address the related laws, rules, and supervisory guidance when deployed inside a global bank like JPMorgan. Specifically, the product needs to pass the requirements on model validation, risk management, AI-tool-use rules, and data protection policies.

The second issue is that **both bankers and clients need to trust our model and estimates**. It not only requires our model to be well tested and validated. The product should be tailored to the client's need and incorporate firm-specific information to make the estimates useful. A lot of work is needed to understand what the real needs of the client are, and how our model can be developed into a specialized tool to meet the need. This requires deeper industry knowledge, professional guidance, and collaboration with clients.

The third issue is **ongoing monitoring and improvement** after launch. Model design and estimates should be updated with new data and customer feedback. We need to develop credible metrics and methods to evaluate the effectiveness of our tool, and quantify its impact.

The plan considers two main jurisdictions: (1) the United States, where JPMorgan is supervised as a US bank, and (2) the Hong Kong SAR, where the bank operates through locally regulated entities.

Scope of the product. For classification purposes, the end product is defined as a quantitative/statistical model that consumes input data and configurations, and outputs statistical estimates. There are several important boundaries:

- The model is not a black box and is fully explainable.
- The model is not a generative model or an agentic system.
- AI-tool-use is restricted to coding implementation and testing, supervised and reviewed by human.

Most of the current AI-related obligations center on generative AI and agentic systems. Our model is designed such that it is not generative or agentic, and it is largely within the standard quantitative/statistical model category.

6.4.1 Key principles of compliance

This section provides a high-level summary of the model risk governance and compliance guidelines in the United States and Hong Kong SAR.

- **A human stays responsible.** Final accountability rests with named people, not with the model. The model is decision support: a banker or officer reviews and signs off, and the model never makes a decision automatically on its own. (US: SR 26-2 governance; HK: HKMA Principle 1, SFC General Principle 9.)
- **No black box.** The model must be explainable to the people who rely on it and to independent reviewers, clearly enough that they can question and challenge it. We have to be able to say, in explicit terms, why the model produced a given number. (US: “effective challenge”; HK: HKMA Principle 3, “no black-box excuse.”)
- **Reproducibility and record-keeping.** The model version, spec, parameters, inputs, and the output that informed any advice must be retained for the applicable period and reproducible on demand. Build versioning and an audit log into the product from the start. Baseline validation should emphasize reproducibility control.
- **Independent validation before it goes live.** The model must be tested and approved before it is used, and the builder cannot also be the validator. (US: SR 26-2 conceptual soundness and effective challenge; HK: HKMA Principle 5, SFC Internal Control Guidelines.)
- **Concrete and complete documentation.** The design, assumptions, model choices, data sources, and test results must be documented well enough for a third party to follow and reproduce the work. In practice, if it is not written down, it does not count as done. (US: SR 26-2 conceptual-soundness documentation; HK: across the AI principles and the conduct code.)
- **Out-of-sample testing.** The model must be checked against actual outcomes on data it was not built on (out-of-sample back-testing), not merely shown to fit its own training data. (US: SR 26-2 outcomes analysis; HK: SPM CA-G-3, validation “should not be limited to back-testing.”)
- **Monitoring after launch.** Performance must be monitored over time, and the model re-checked or re-estimated when markets or conditions change. (US: SR 26-2 ongoing monitoring; HK: HKMA AI principles.)
- **Be honest about limitations.** Known weaknesses must be disclosed to users, and use restricted where the model is weak. For us this means reporting ranges rather than false precision where parameters are only weakly identified. (US: SR 26-2 on use limitations; HK: same expectation.)
- **Input data quality.** The data feeding the model must be appropriate and of good quality, and the data choices documented. (US: SR 26-2 data selection; HK: HKMA Principle 4, data quality.)
- **If sold, give the buyer enough to validate it.** A buyer institution stays accountable for any model it uses, so we must ship a transparency package that lets its own reviewers

understand and check the model without us handing over proprietary code. (US: SR 26-2, Section VII; HK: third-party provider principles.)

Appendix A

Regulatory Compliance Map

Scope. This appendix summarizes the laws, rules, and supervisory guidance that a mature version of the product must address when deployed inside a global bank like JPMorgan, across two jurisdictions: the United States, where JPMorgan is supervised as a US bank, and the Hong Kong SAR, where it operates through locally regulated entities. Because the properly-implemented product is a standard statistical and quantitative model rather than a generative AI tool, the governing regime in each jurisdiction is the **quantitative-model (model-risk) regime**, with the AI-specific rules a short overlay.

What “our product” is, for classification purposes. The end product is built mostly on the **DF26 framework**, a deep-learning approach to structural estimation, with classical methods (VFI / PFI for solving the model, SMM and Bayesian inference for estimating parameters) as benchmarks. Every method in the stack turns input data into quantitative estimates, and **none is a generative model or an agentic system**. That single fact places the product in the ordinary quantitative-model lane, not the generative-AI lane, in both jurisdictions.

Use of coding assistants. The code is written with help from a general coding assistant, but the algorithm and product design are human-owned and the assistant only implements human-specified, human-reviewed plans. This does not change the model’s classification; it adds two ordinary controls: all AI-assisted code must be reviewed and tested against the human design, and what may be sent to an external assistant is governed by the bank’s AI-tool-use and data-protection policies.

As-of date. Verified against sources available as of 17 June 2026.

A.1 The shared compliance baseline and key takeaways

Both the US and Hong Kong regimes demand the same core baseline of a quantitative model; build the model and its paperwork to satisfy these once, and it meets both sides. The same baseline is set out, with jurisdiction sources, in Chapter 1 (Model risk and compliance).

- **A human stays responsible.** The model is decision support; a banker or officer reviews and signs off, and it never decides on its own.
- **No black box.** The model must be explainable to those who rely on it and to independent reviewers, clearly enough to question and challenge.
- **Independent validation before it goes live.** The model must be tested and approved before use, and the builder cannot also be the validator.

- **Write everything down.** Design, assumptions, model choices, data sources, and test results must be documented well enough for a third party to reproduce the work.
- **Test against the real world, not just the fit.** The model must be checked against actual outcomes on data it was not built on, not merely shown to fit its training data.
- **Keep watching it after launch.** Performance must be monitored over time, and the model re-checked or re-estimated when conditions change.
- **Be honest about limitations.** Known weaknesses must be disclosed and use restricted where the model is weak, reporting ranges rather than false precision.
- **Mind the input data.** The data feeding the model must be appropriate, of good quality, and documented.
- **If sold, give the buyer enough to validate it.** Ship a transparency package that lets the buyer’s reviewers check the model without handing over proprietary code.

Match the rigor to the stakes. Everything above scales with materiality: a low-stakes advisory input a banker double-checks needs lighter treatment than a model wired into capital or pricing. This is why the product is designed to be deployed as a reviewed advisory input rather than an automated engine. A single validation-and-governance package, built to the SR 26-2 standard, satisfies the whole baseline for both jurisdictions.

Key takeaways:

1. **Packaging, not substance.** The US states the whole regime in one document (SR 26-2); Hong Kong assembles the same obligations from several instruments. The cost is mapping and documentation, not a different set of controls.
2. **The advisory conduct layer is separate and applies in both places.** US SEC / FINRA rules and HK SFC rules govern how the model’s output is used in client advice; the common thread is a human adviser who remains responsible.
3. **The generative-AI rules are an overlay we have engineered around.** Because the product is non-generative, it stays in the ordinary quantitative-model lane; only adding a generative or agentic feature would change that.

A.2 Consolidated list

Jurisdiction and issuer	Instrument	Type	Binds our product?
US — Fed / OCC / FDIC	SR 26-2 / OCC Bulletin 2026-13, Revised MRM Guidance	Quantitative-model (model risk)	Yes
US — SEC / FINRA	SEC / FINRA supervision, recordkeeping, fair dealing	Advisory conduct overlay	Conditional (client-facing advice)
US — Fed / OCC / FDIC	Interagency Third-Party Risk Mgmt Guidance (OCC 2023-17)	Vendor onboarding	Yes, if we are the vendor
US — Fed / OCC / FDIC; White House	Forthcoming AI RFI; EO 14179 federal posture	AI-specific overlay	No (watch only)
HK — HKMA; Legislature; PRA (benchmark)	Banking Ordinance Cap. 155 (7th Sch.); SPM IC-1, CG-1, CA-G-4, CA-G-3; HKMA 2019 AI principles; PRA SS1/23 (benchmark)	Quantitative-model (model risk)	Yes (bank entity)
HK — SFC	Code of Conduct (GP1, 2, 3, 5, 7, 9); CFA Code (Type 6); Internal Control Guidelines	Advisory conduct + model governance	Yes (advisory entity)

Jurisdiction and issuer	Instrument	Type	Binds our product?
HK — Legislature / PCPD	PDPO (Cap. 486) + PCPD AI Model Framework	Data privacy	Yes, if personal data (CEO product)
HK — SFC; HKMA	SFC GenAI circular (24EC55); HKMA GenAI consumer circular	AI-specific overlay	No (generative / customer-facing only)

A.3 Watch list

1. **US interagency AI RFI.** Not yet issued; will define the regime for generative and agentic AI. Relevant only if we add such a component.
2. **HKMA updates to the 2019 AI principles.** Expected to be updated over time; track for any move toward firmer model requirements.
3. **A dedicated HK model-risk standard.** Given international movement (PRA SS1/23) and active HKMA supervision, a more explicit local expectation could emerge; track HKMA SPM updates.
4. **HK GenAI Sandbox / Sandbox++.** A supervised testing route that could be a channel to pilot the product in Hong Kong; an option, not an obligation.

References

- Victor Duarte and Julia Fonseca. AI for structural estimation. Technical report, National Bureau of Economic Research, 2026.
- Victor Duarte, Diogo Duarte, and Dejanir H Silva. Machine learning for continuous-time finance. *The Review of Financial Studies*, 37(11):3217–3271, 2024.
- Jesús Fernández-Villaverde. Deep learning for solving economic models. Working Paper 34250, National Bureau of Economic Research, September 2026. URL <http://www.nber.org/papers/w34250>.
- Xiaodan Gao, Toni M Whited, and Na Zhang. Corporate money demand. *The Review of Financial Studies*, 34(4):1834–1866, 2021.
- Ronald A Howard. *Dynamic programming and markov processes*. John Wiley, 1960.
- Lilia Maliar, Serguei Maliar, and Pablo Winant. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101, 2021.
- Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Ilya A Strebulaev and Toni M Whited. Dynamic Models and Structural Estimation in Corporate Finance. *Foundations and trends in finance*, 2012.
- Chak Wong. Solving a Risky-Debt Corporate Finance Model with Deep Learning. Technical report, Working paper, 2026.

Appendix: Interview QA

The interview questions and answers follow as a separately appended PDF.